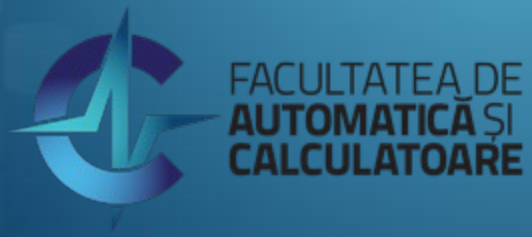


Arhitecturi Paralele
Abordări probleme paralele

Radu-Ioan CIOBANU
radu.ciobanu@upb.ro

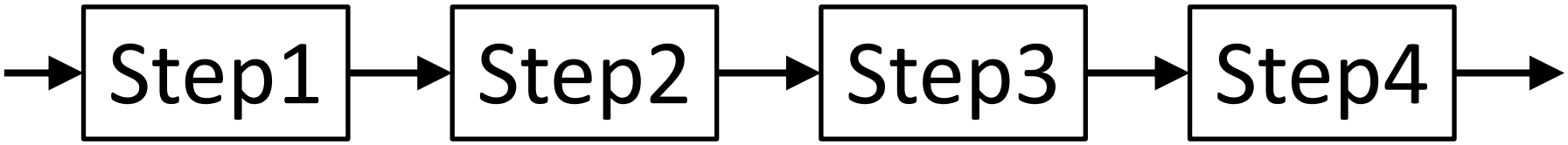




Paralelism: modele Pipeline

Pipeline

- CPU Instruction pipeline
- Graphics pipeline
- Various algorithms



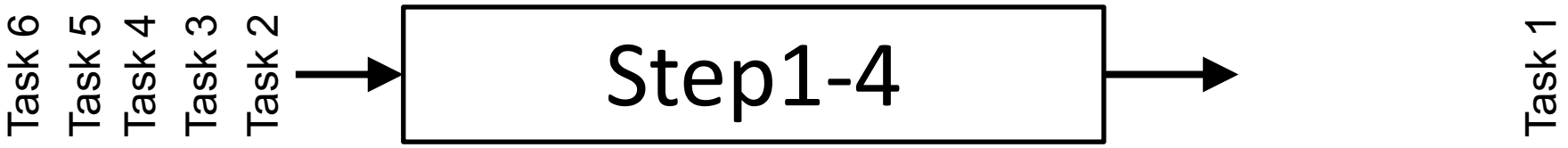
Un pas poate fi executat de:

- thread
- process
- element hardware

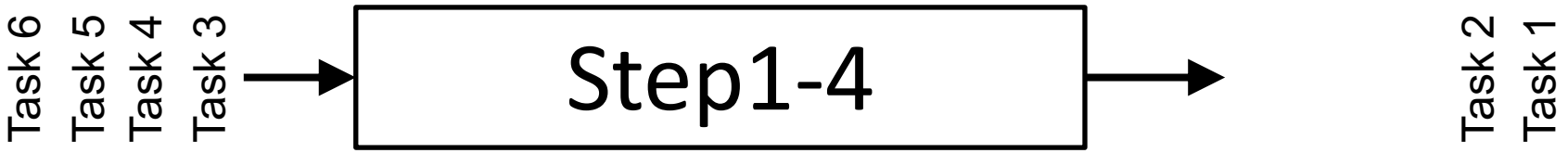
Without Pipeline



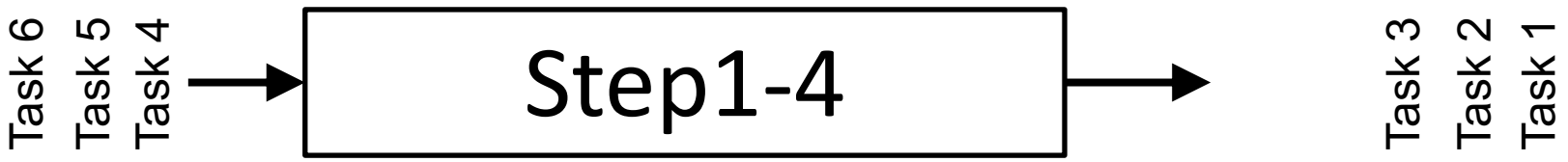
Without Pipeline



Without Pipeline



Without Pipeline



Without Pipeline



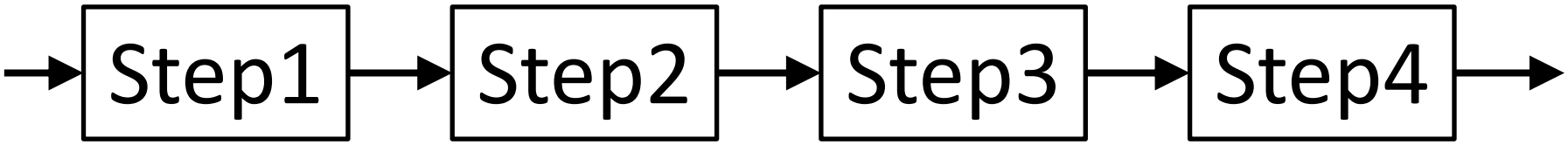
Without Pipeline



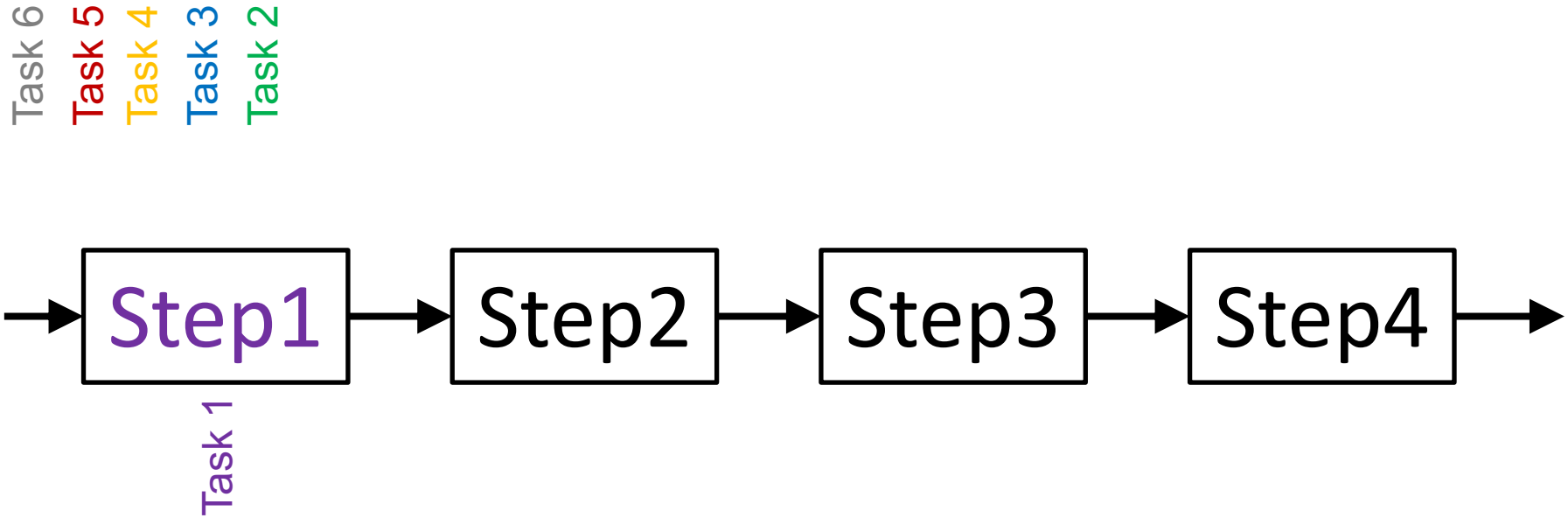
$$total_execution_time = task_execution_time * number_of_tasks$$

Pipeline

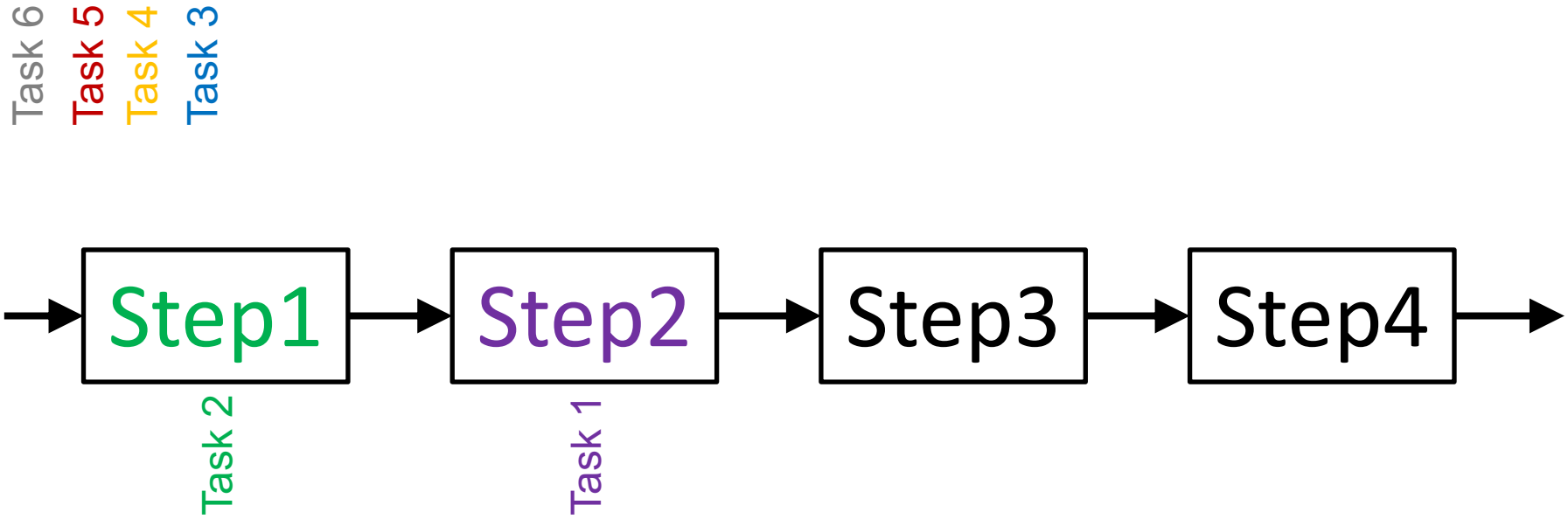
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1



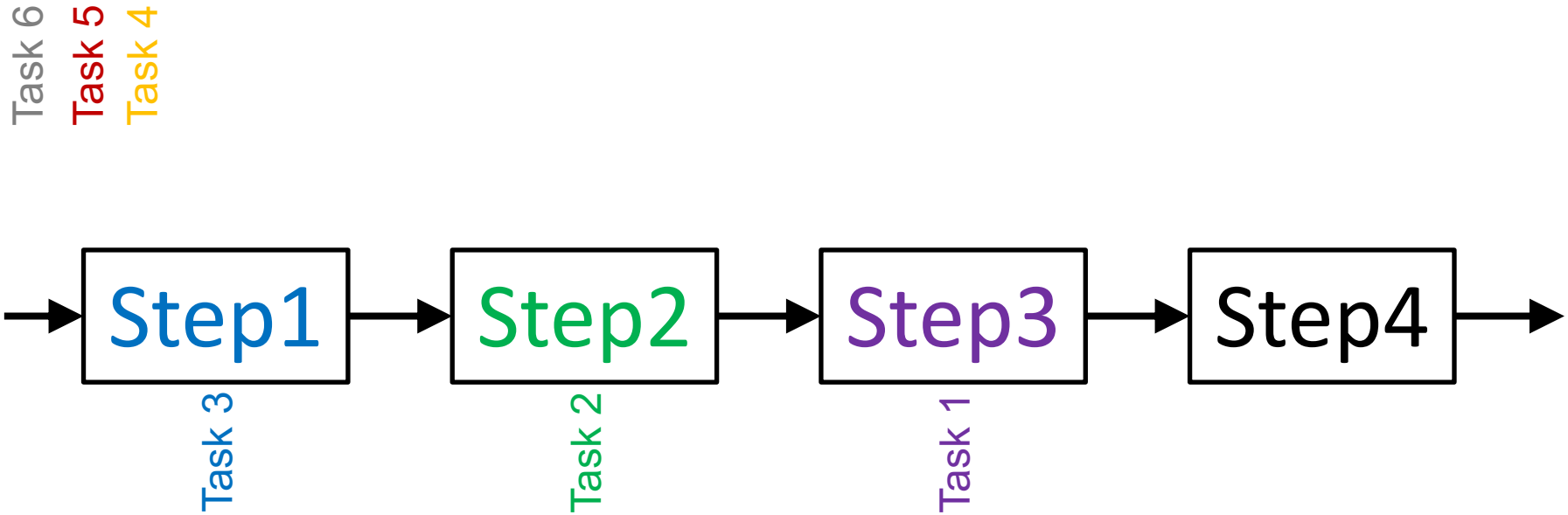
Pipeline


Task 6
Task 5
Task 4
Task 3
Task 2
Step1
Step2
Step3
Step4
Task 1

Pipeline


Task 6
Task 5
Task 4
Task 3
Step1
Step2
Step3
Step4
Task 2
Task 1

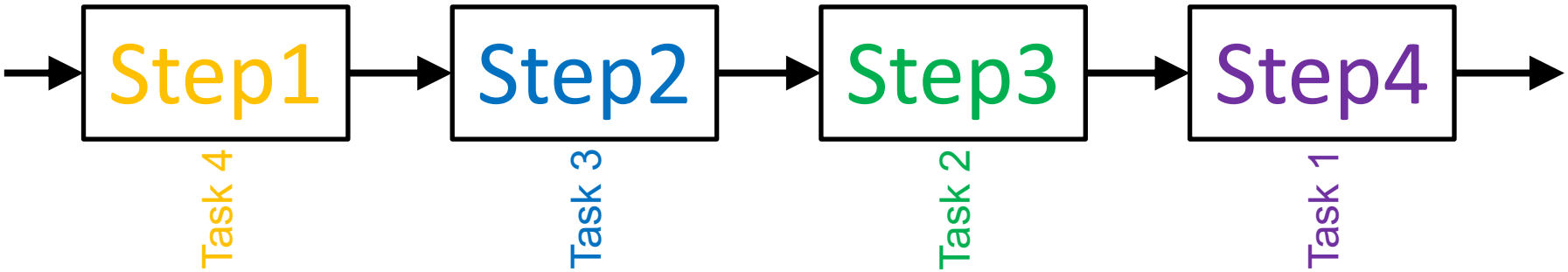
Pipeline


Task 6
Task 5
Task 4
Step1
Step2
Step3
Step4
Task 3
Task 2
Task 1

Pipeline

Task 6

Task 5



Task 4

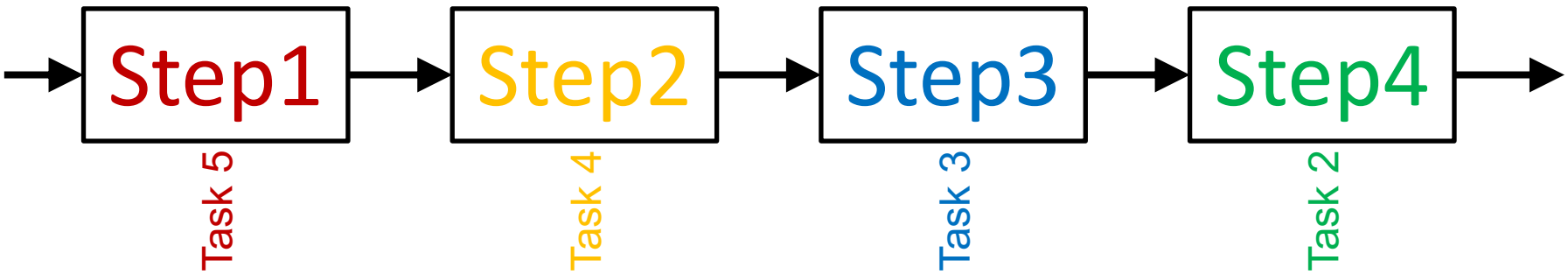
Task 3

Task 2

Task 1

Pipeline

Task 6



Task 5

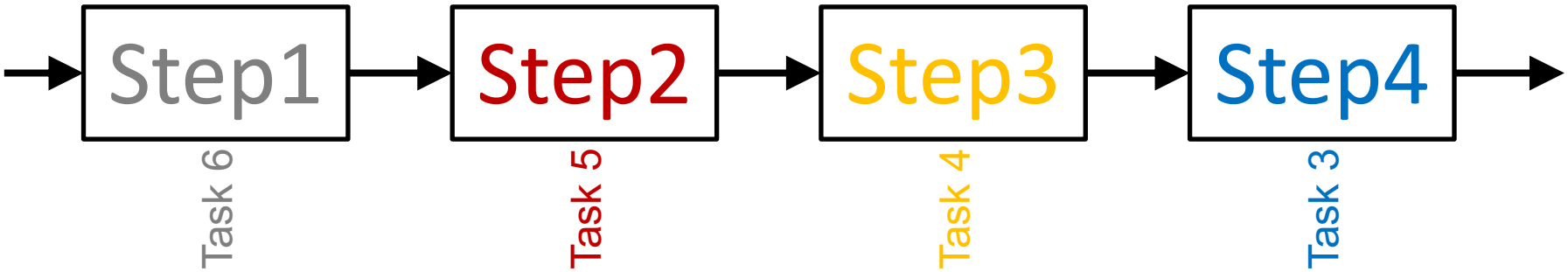
Task 4

Task 3

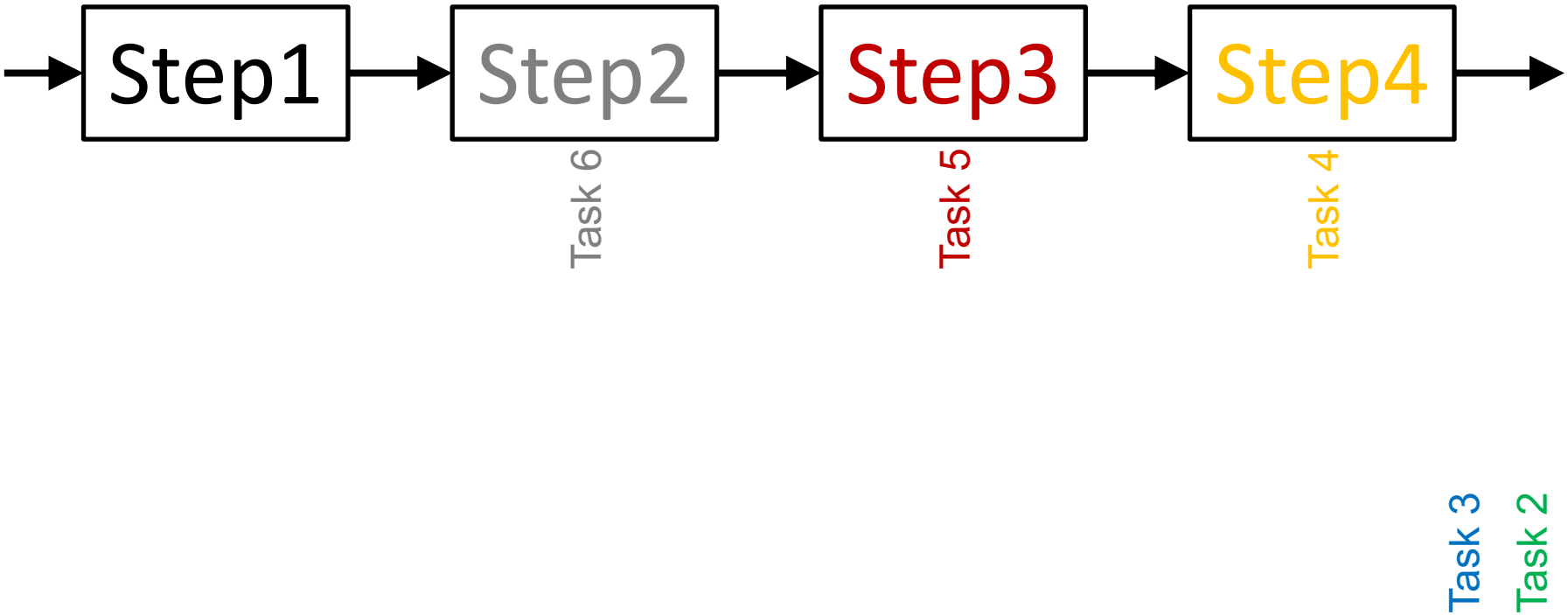
Task 2

Task 1

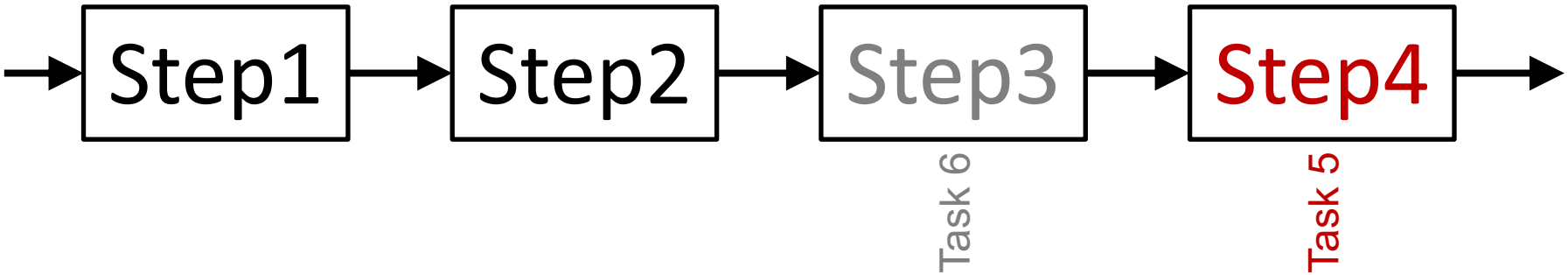
Pipeline


Step1
Step2
Step3
Step4
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1

Pipeline


Step1
Step2
Step3
Step4
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1

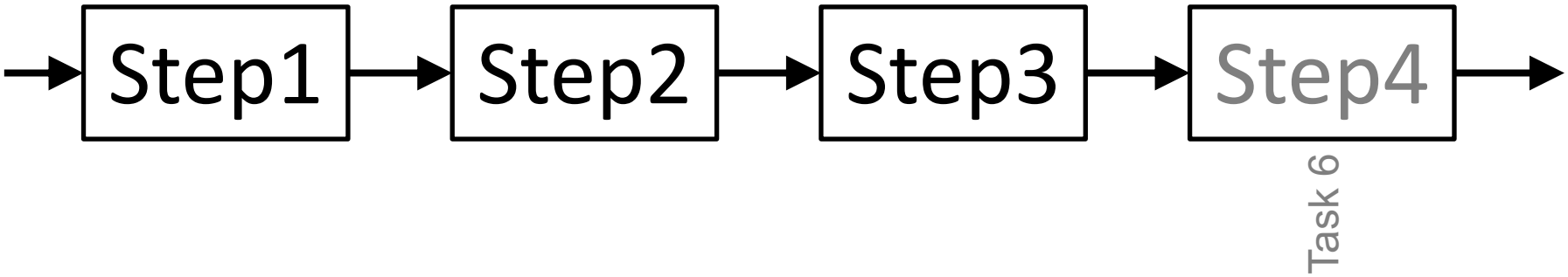
Pipeline



Task 4
Task 3
Task 2
Task 1

Pipeline

Ideal: $step_execution_time = \frac{task_execution_time}{number_of_steps}$



După $number_of_steps$ taskuri:

$total_execution_time = number_of_tasks * step_execution_time$

Un task se termină la fiecare “step tick”



Sortare...

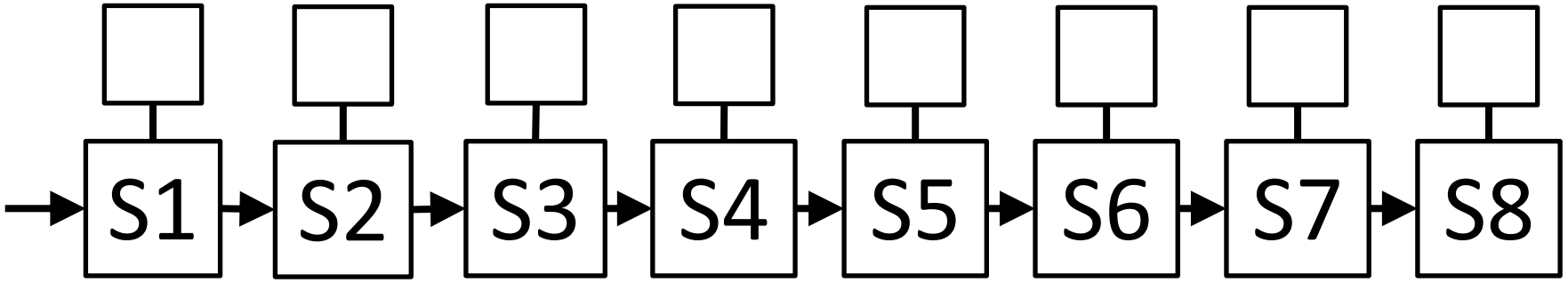
Sorting

9	4	2	7	6	5	6	1

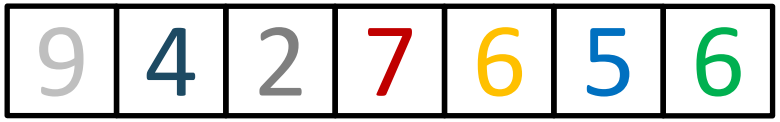
1	2	4	5	6	6	7	9

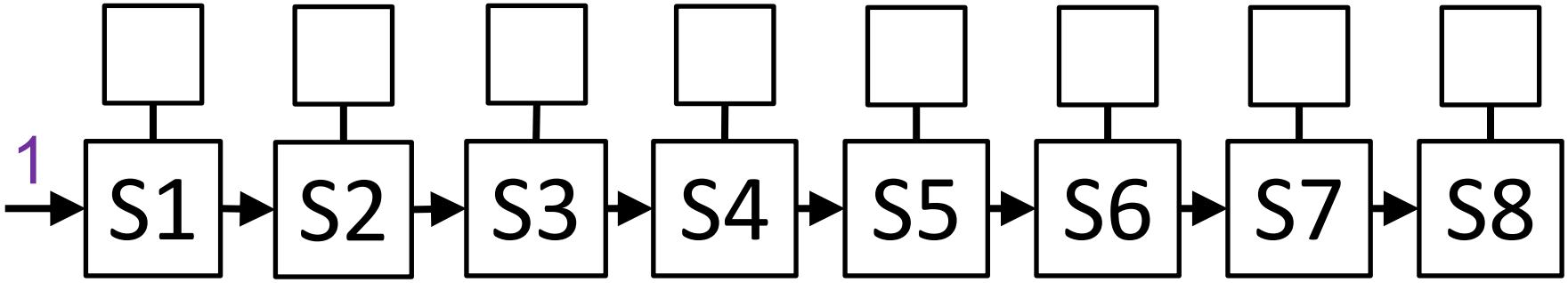
Sorting with pipeline



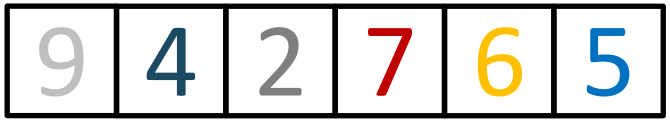


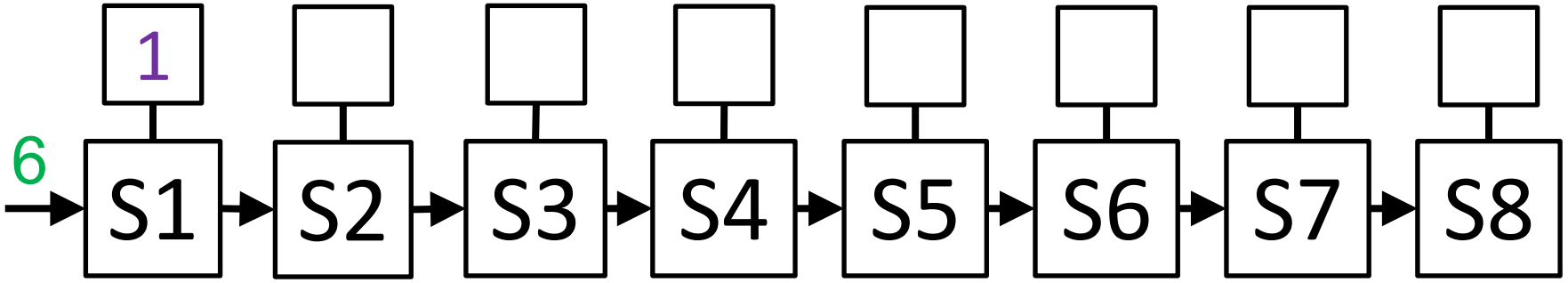
Sorting with pipeline



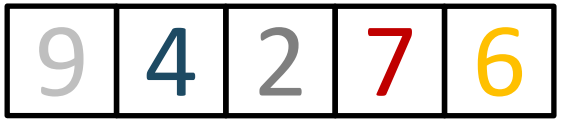


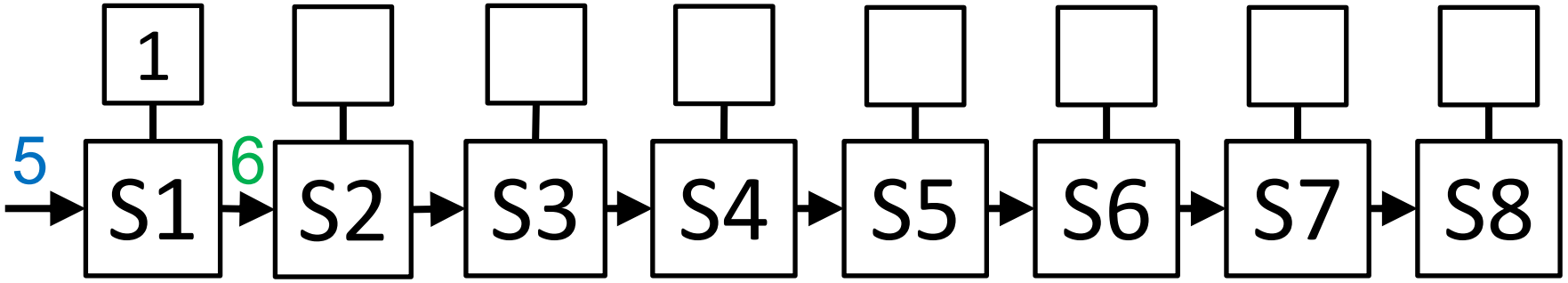
Sorting with pipeline





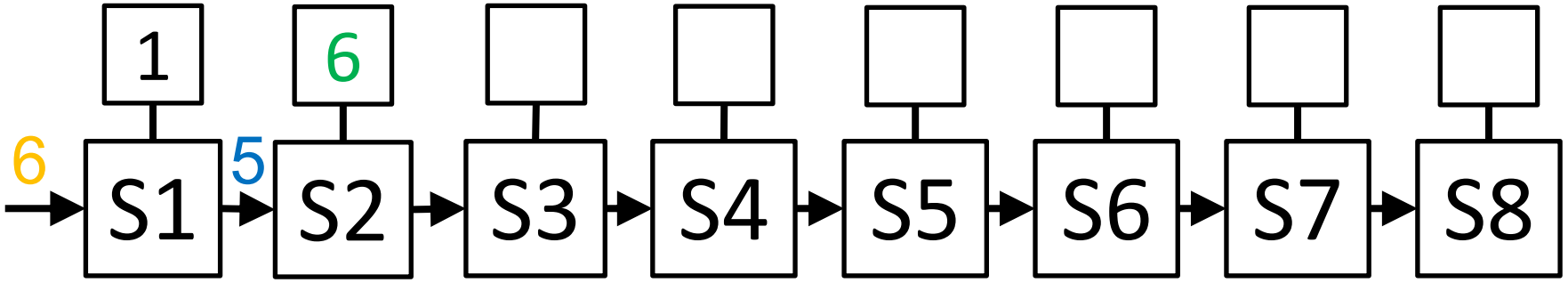
Sorting with pipeline





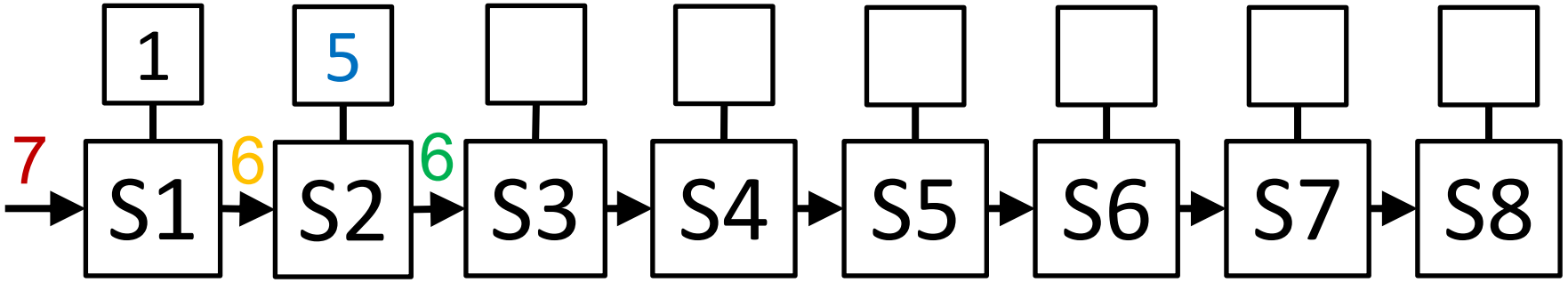
Sorting with pipeline





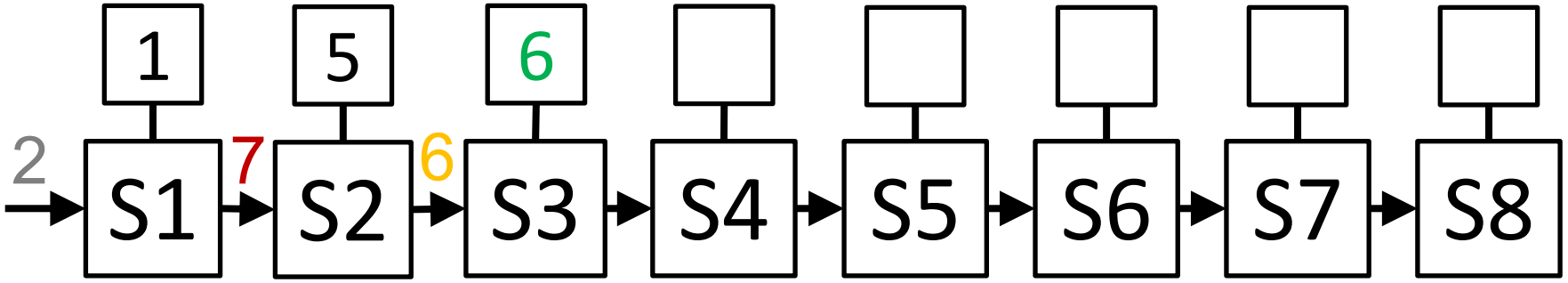
Sorting with pipeline



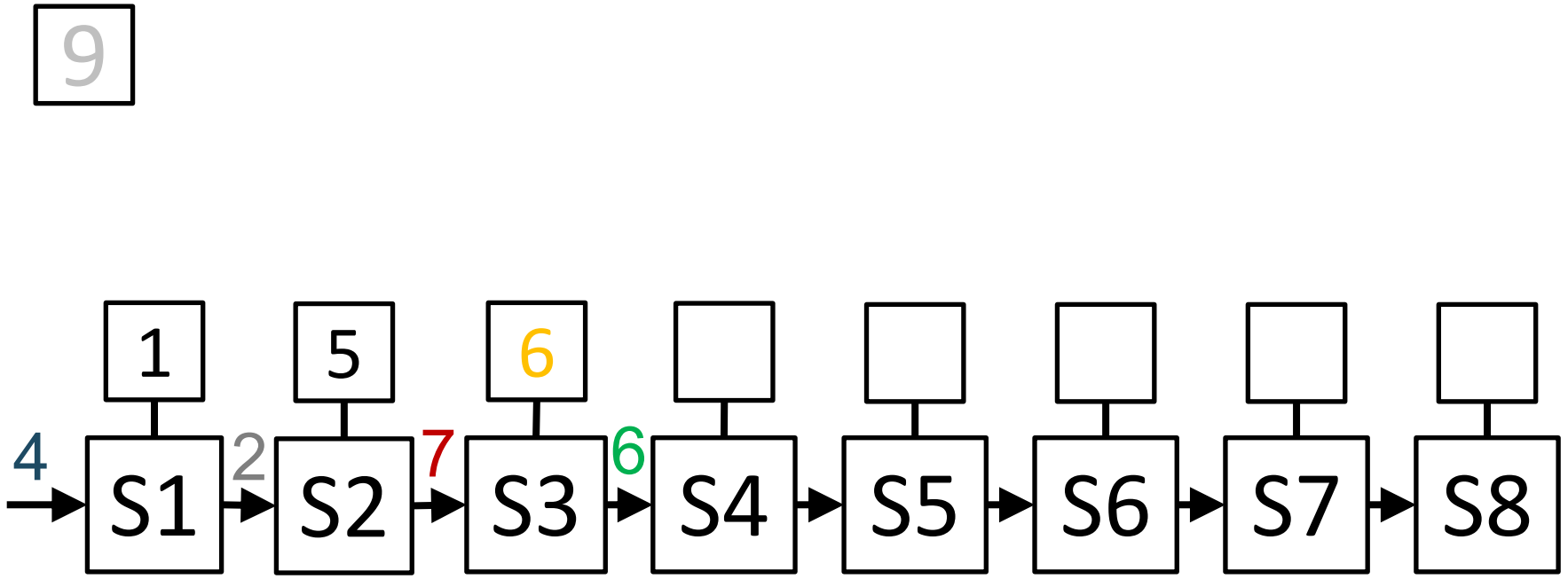


Sorting with pipeline

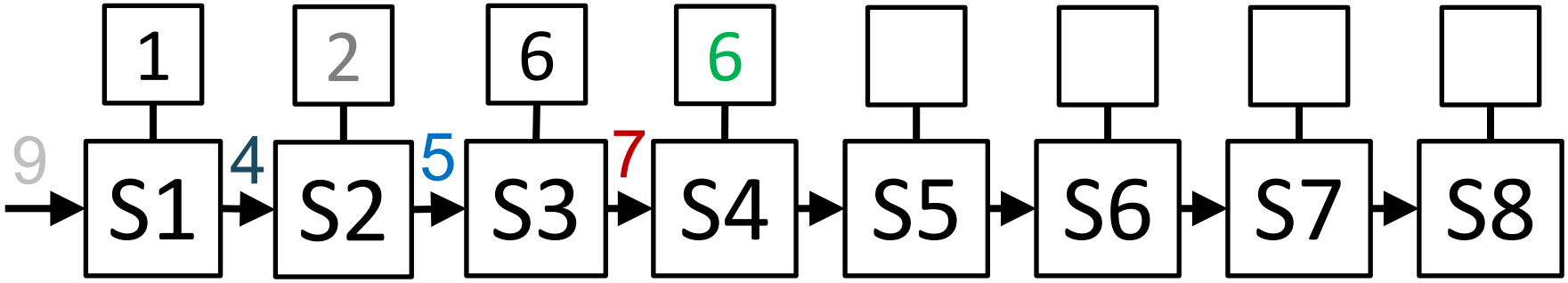




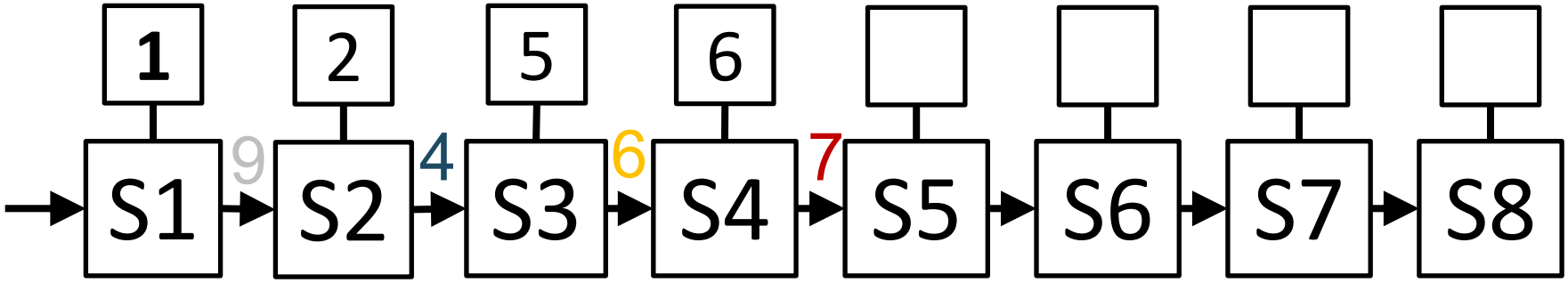
Sorting with pipeline



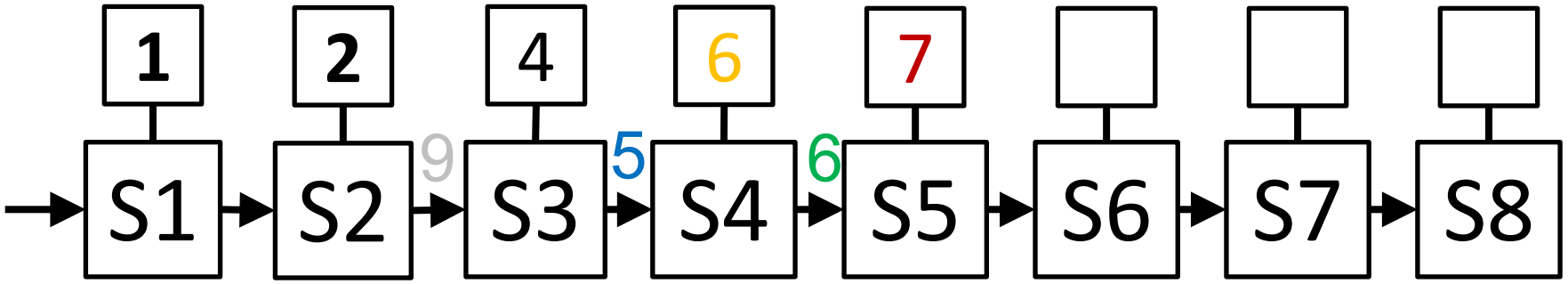
Sorting with pipeline



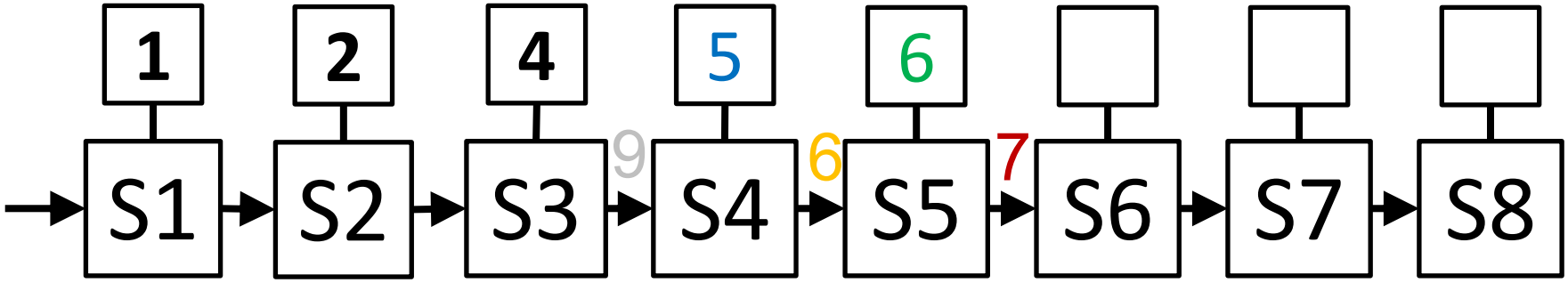
Sorting with pipeline



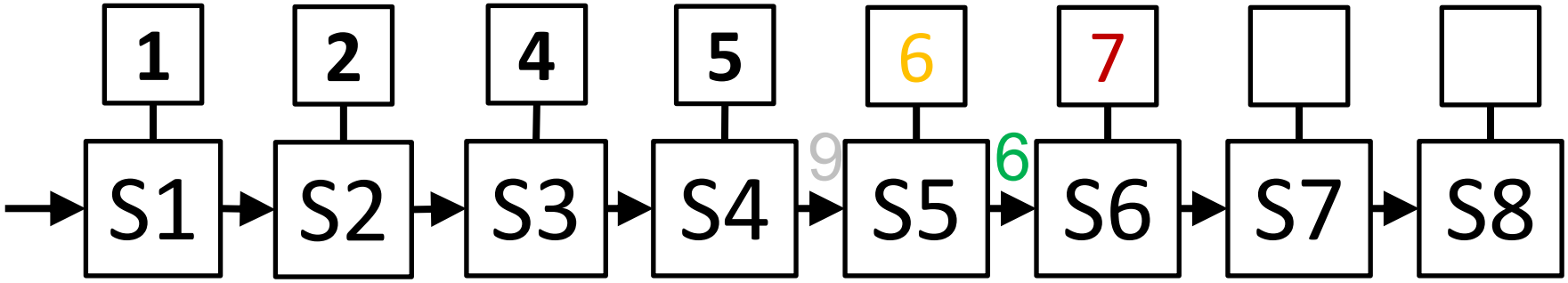
Sorting with pipeline



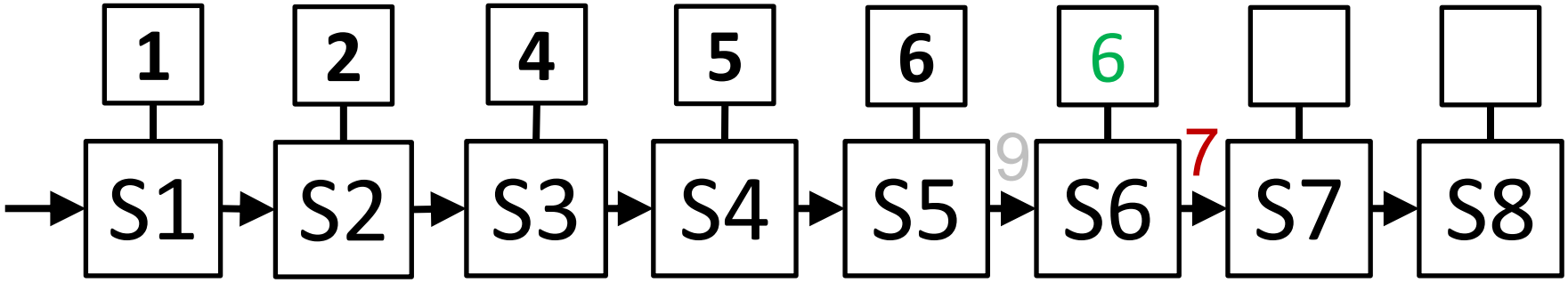
Sorting with pipeline



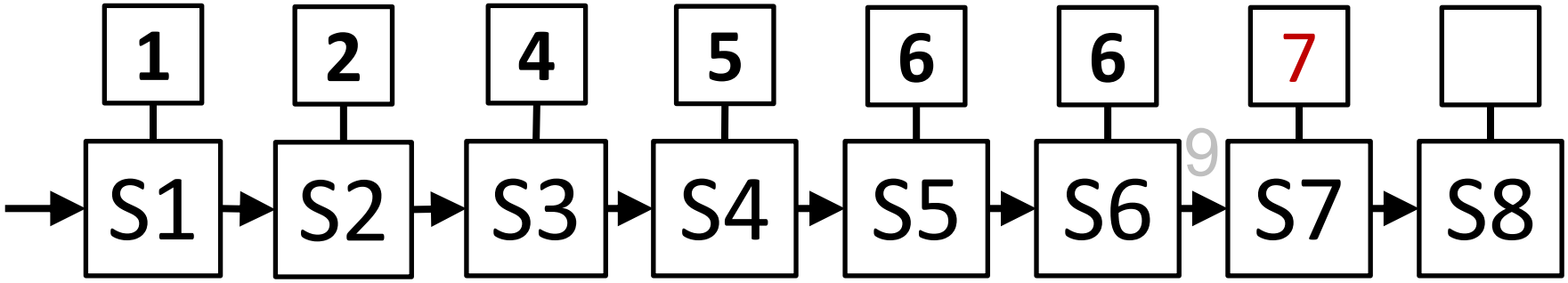
Sorting with pipeline



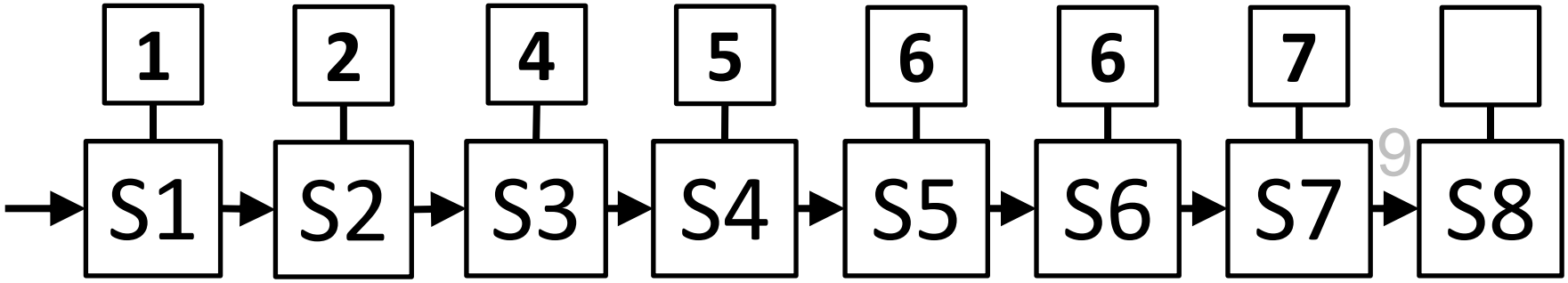
Sorting with pipeline



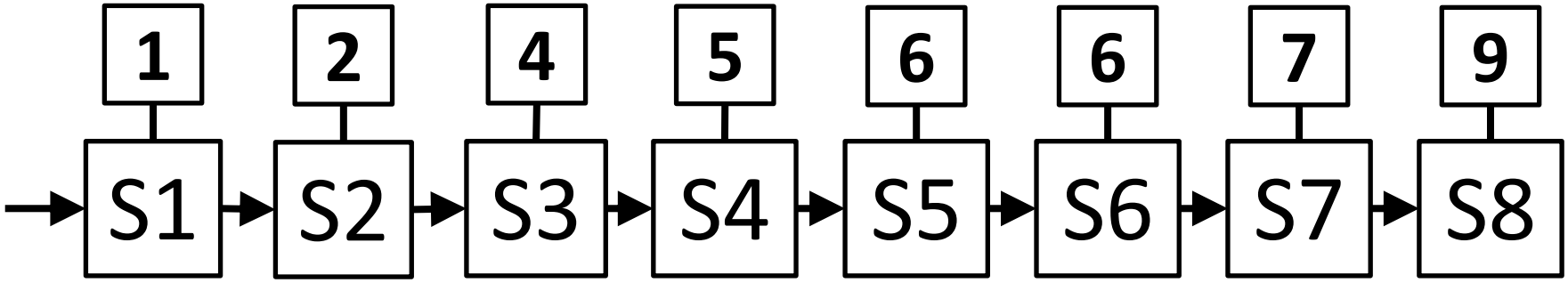
Sorting with pipeline



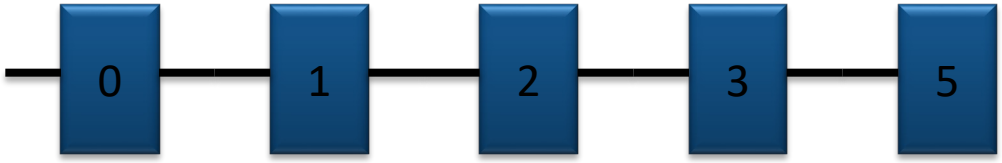
Sorting with pipeline



Sorting with pipeline



Pașii algoritmului de sortare



Pentru **N** valori, **faza 1** durează **2N-1** pași

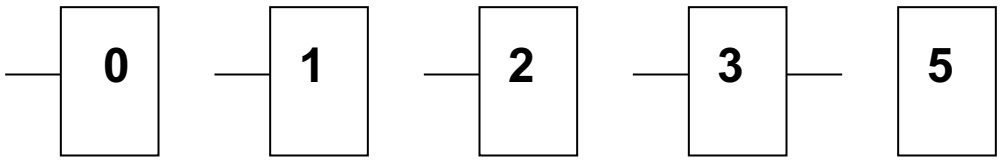
Faza a doua

- Valorile ordonate sunt scoase prin celula din stânga
- Variante:
  1. fiecare procesor începe să transmită spre stânga, imediat ce numerele sunt sortate;
  2. cunoscând poziţia sa din vector şi numărând valorile inspectate, fiecare procesor calculează momentul când începe să transmită spre stânga;
  3. procesorul din dreapta începe să transmită spre stânga imediat ce primeşte o valoare; toate celelalte încep să transmită spre stânga imediat ce primesc o valoare din dreapta;
  4. fiecare procesor începe să transmită spre stânga imediat ce nu mai primeşte o valoare dinspre stânga.

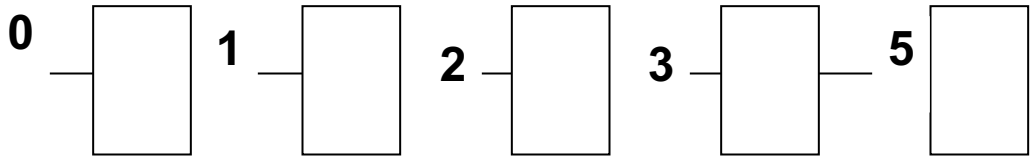
- Complexitate:
  - metoda 3 → **4N-3** pași
  - metoda 4 → **3N-2** pași

- Variantele 1 și 2 impun cunoașterea
  - Pozitiei procesoarelor in vector
  - Contorizarea valorilor

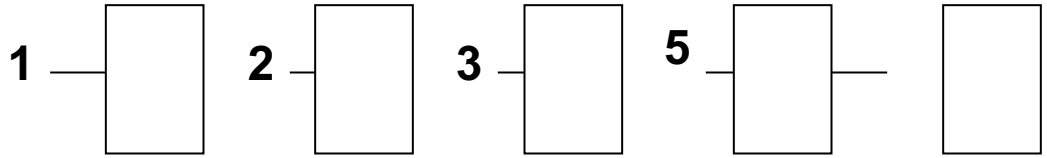
Metoda 4 faza 2



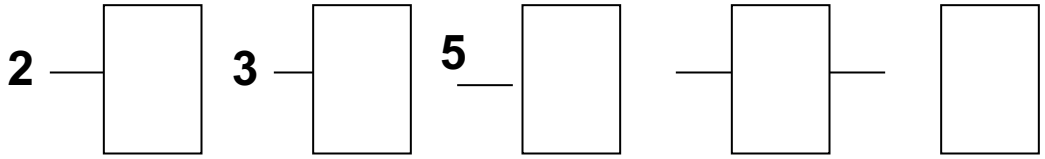
sfarsit faza 1

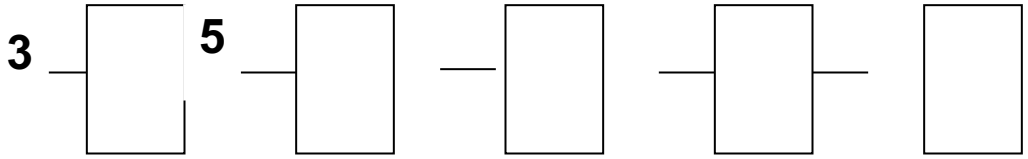


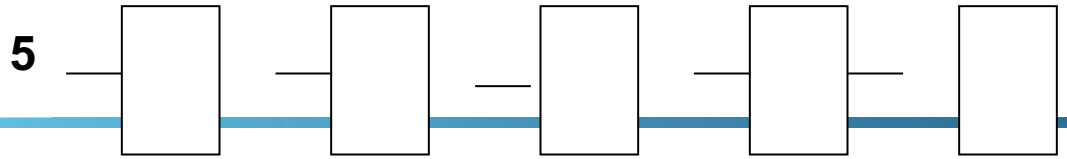
un pas - procesoarele observa ca nu mai primesc din stanga si transmit in stanga



un pas – valoarea din dreapta trece la iesirea din stanga





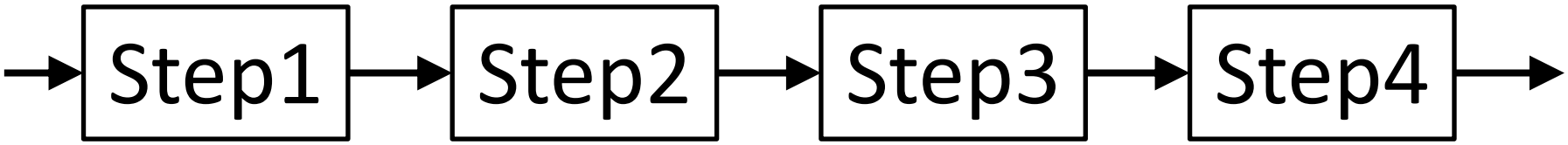


Calcule polinomiale...

Polynomial

$$P(x) = \sum_{i=0}^{n} a_i x^i = a_0 x^0 + a_1 x^1 + a_2 x^2 + \cdots + a_{n-1} x^{n-1} + a_n x^n, n \geq 0$$

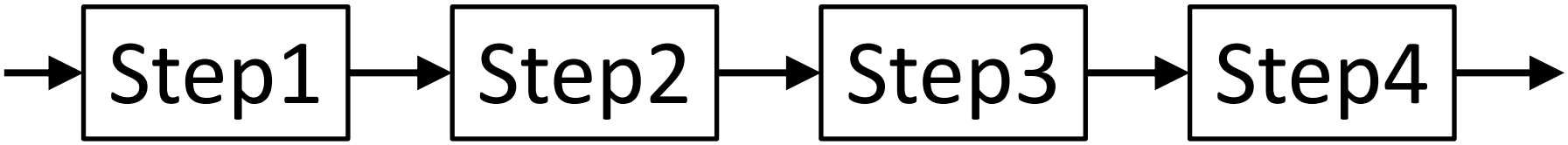
Pipeline



Polynomial calculation + Pipeline

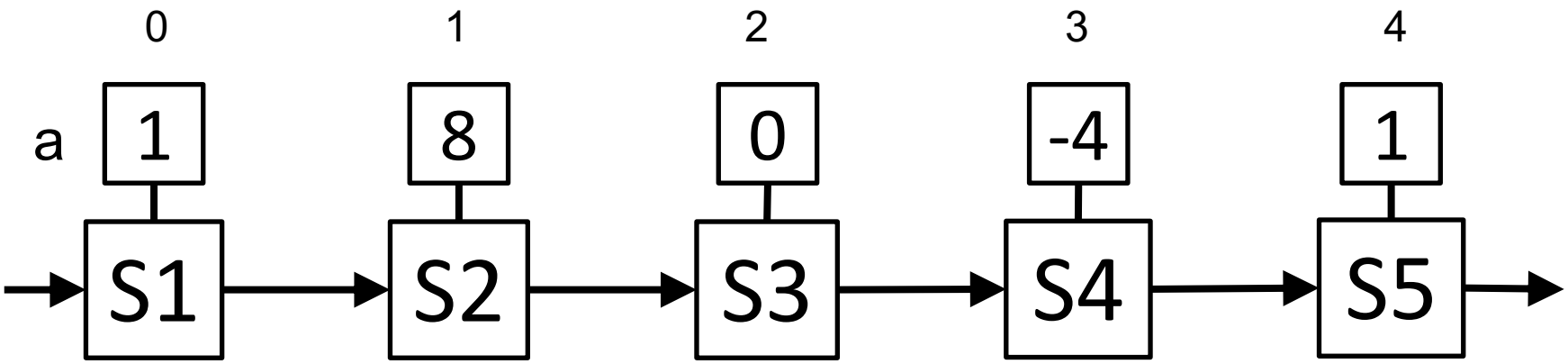
$$P(x) = 1 + 8x + (-4)x^3 + x^4$$

	0	1	2	3	4
a	1	8	0	-4	1



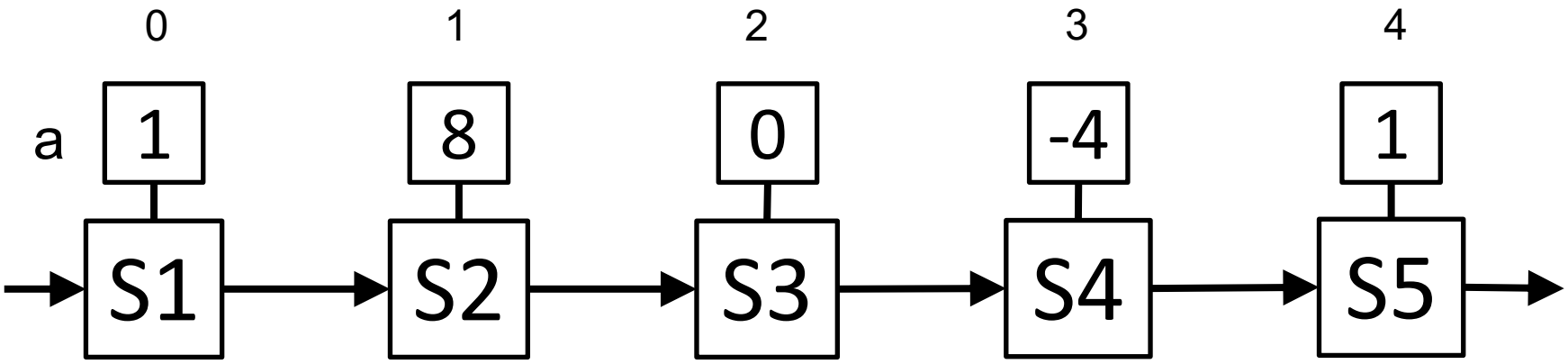
Polynomial calculation + Pipeline

$$P(x) = 1 + 8x + (-4)x^3 + x^4$$



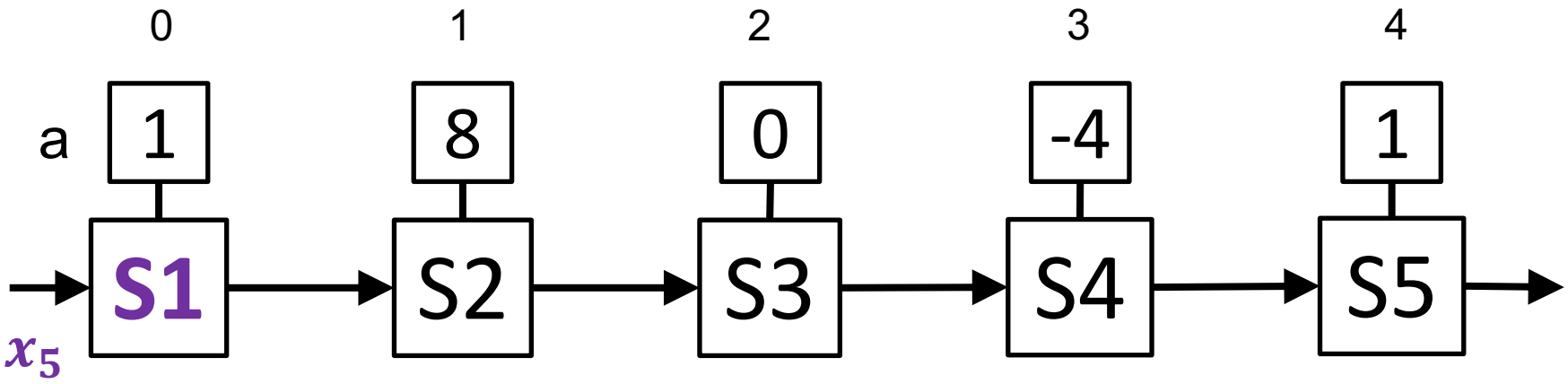
Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4 x_5



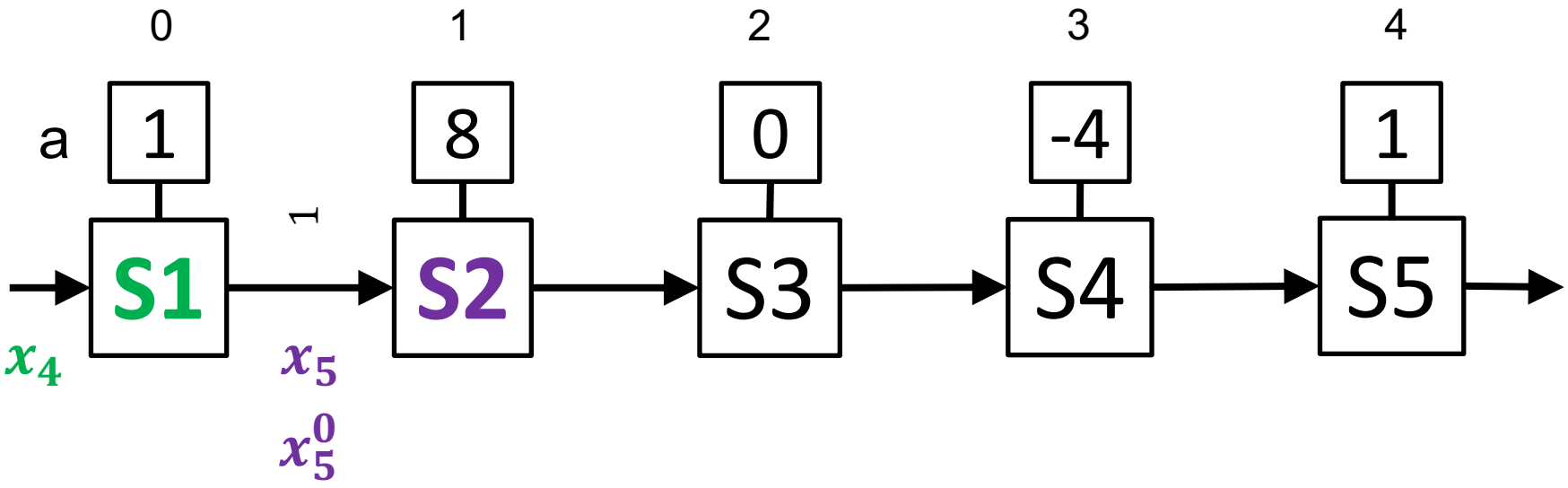
Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4



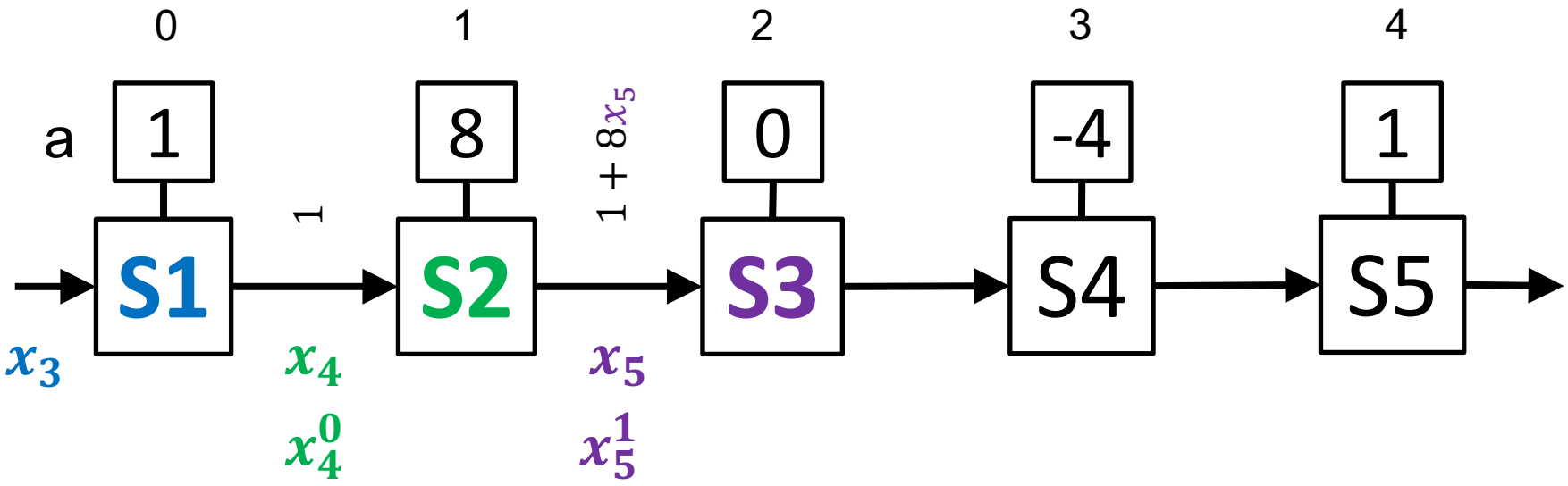
Polynomial calculation + Pipeline

x_1 x_2 x_3



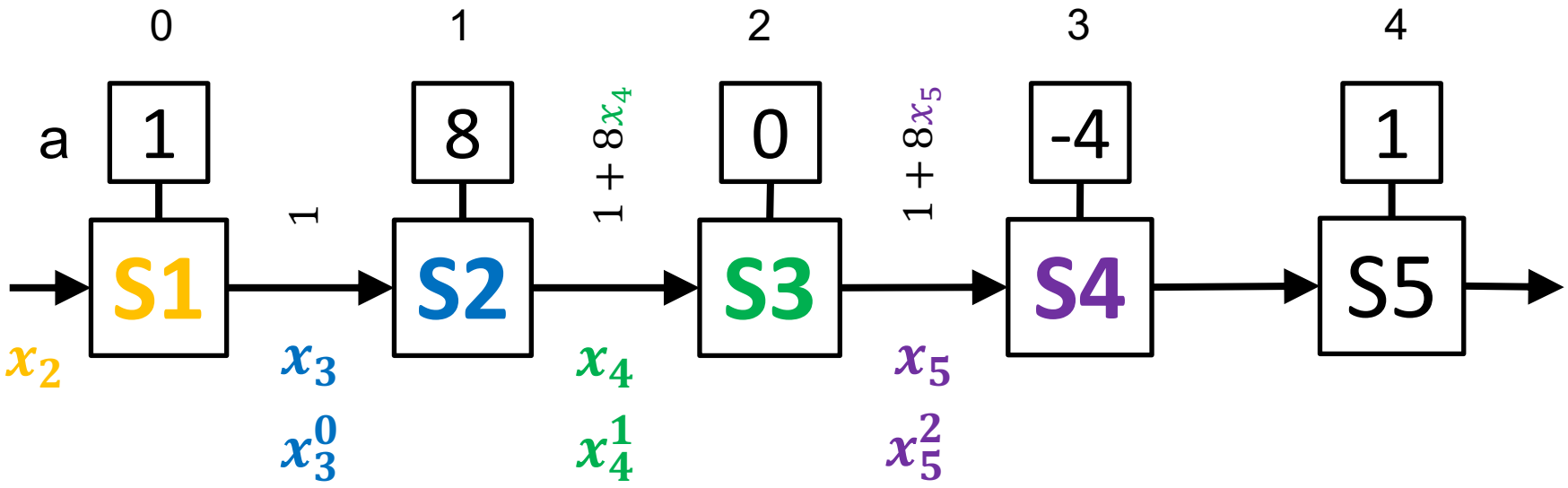
Polynomial calculation + Pipeline

x_1 x_2

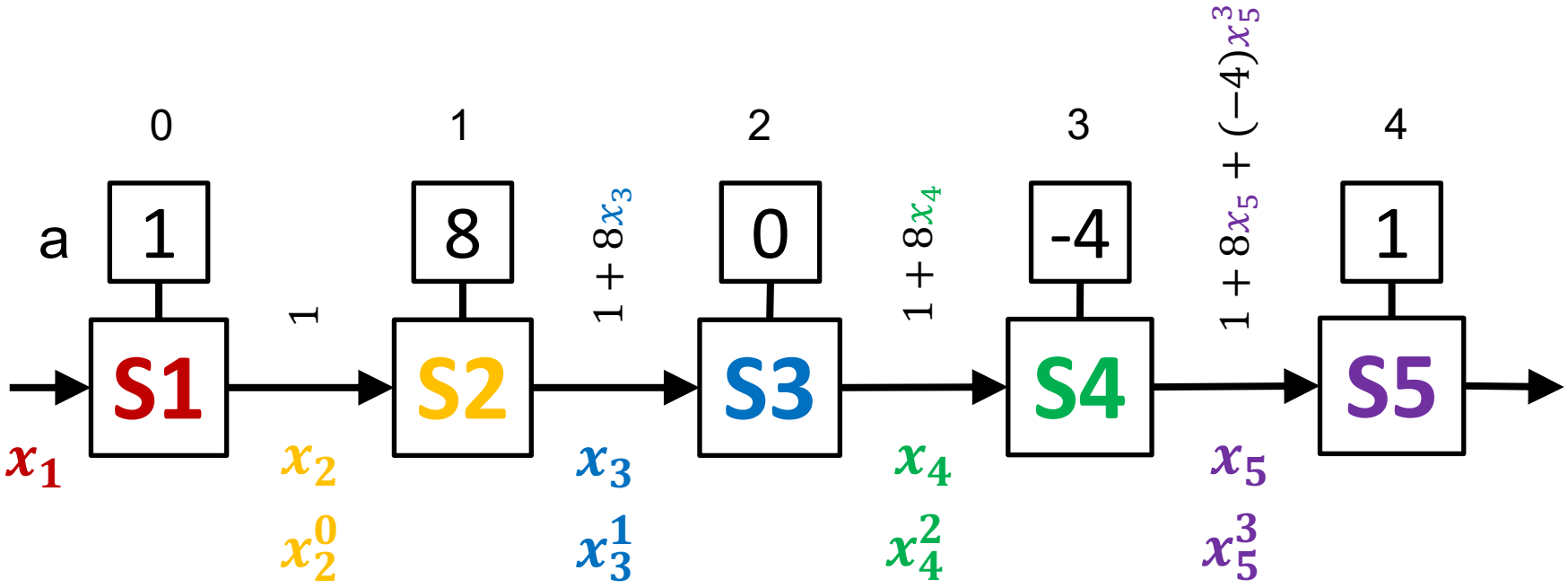


Polynomial calculation + Pipeline

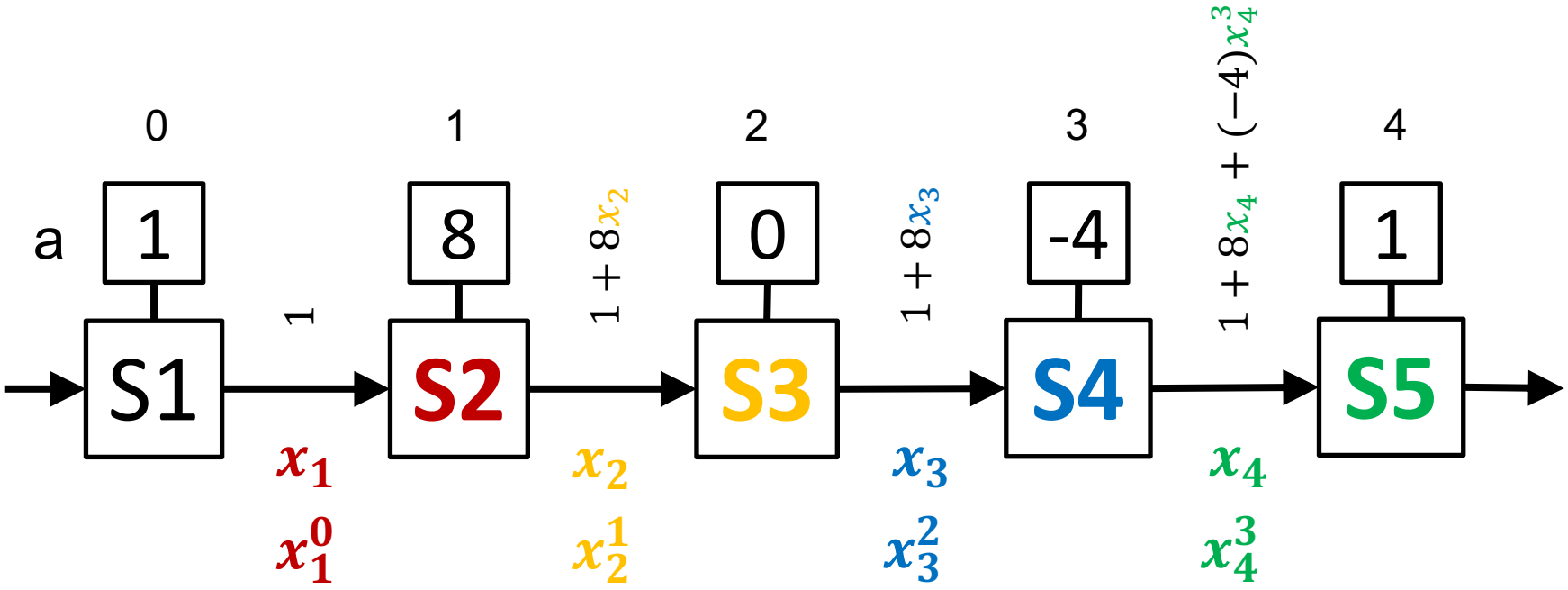
x_1



Polynomial calculation + Pipeline

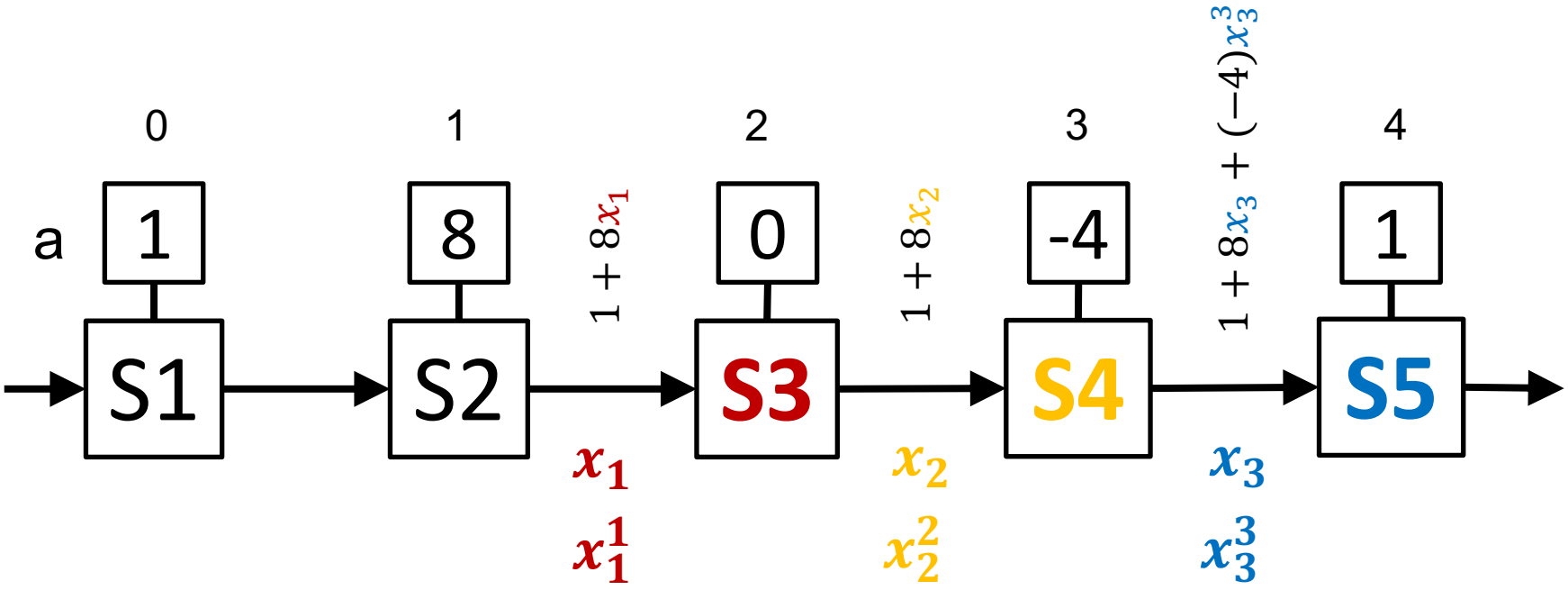


Polynomial calculation + Pipeline



$$\mathbf{1+8}x_5+\mathbf{(-4)}x_5^3+x_5^4$$

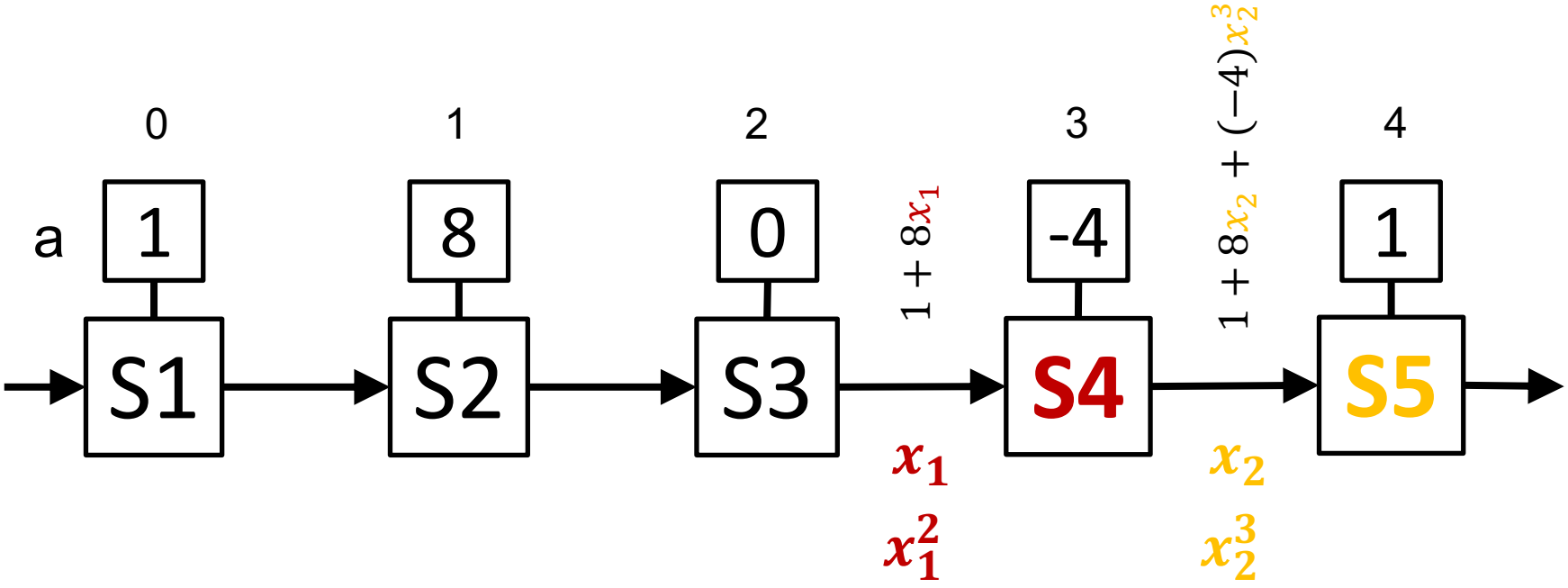
Polynomial calculation + Pipeline



$$1+8x_4+(-4)x_4^3+x_4^4$$

$$1+8x_5+(-4)x_5^3+x_5^4$$

Polynomial calculation + Pipeline

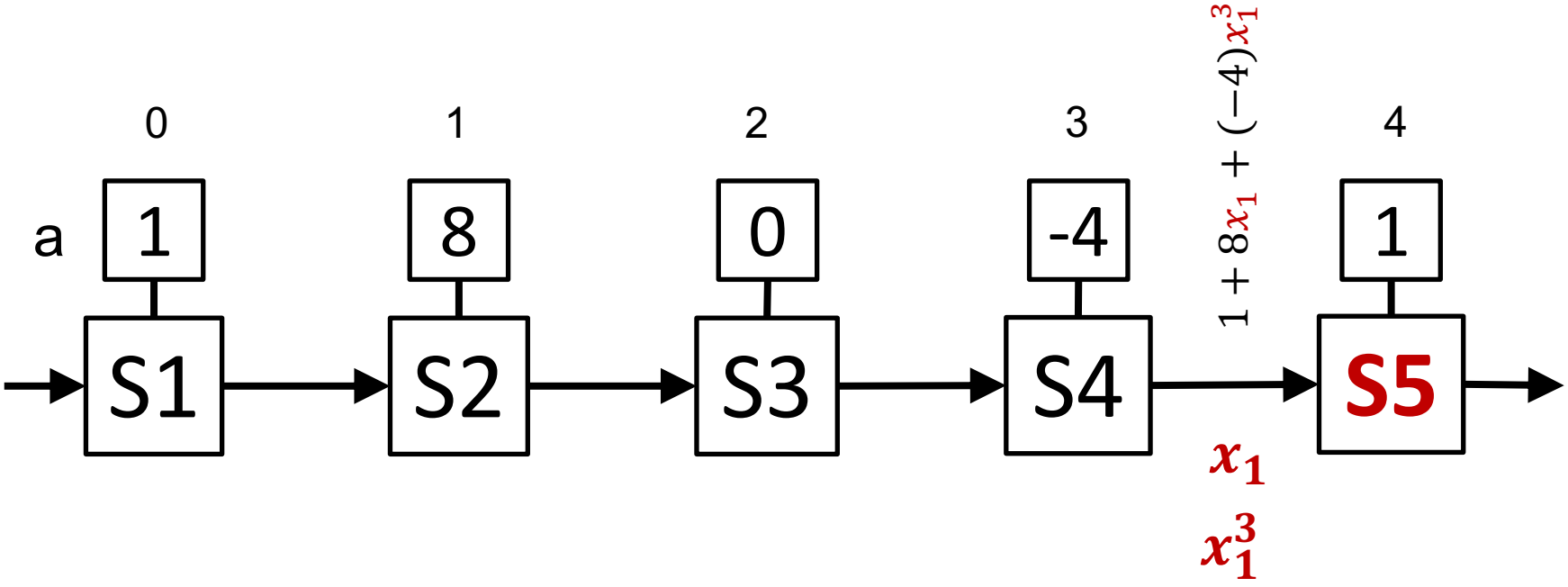


$$1 + 8x_3 + (-4)x_3^3 + x_3^4$$

$$1 + 8x_4 + (-4)x_4^3 + x_4^4$$

$$1 + 8x_5 + (-4)x_5^3 + x_5^4$$

Polynomial calculation + Pipeline



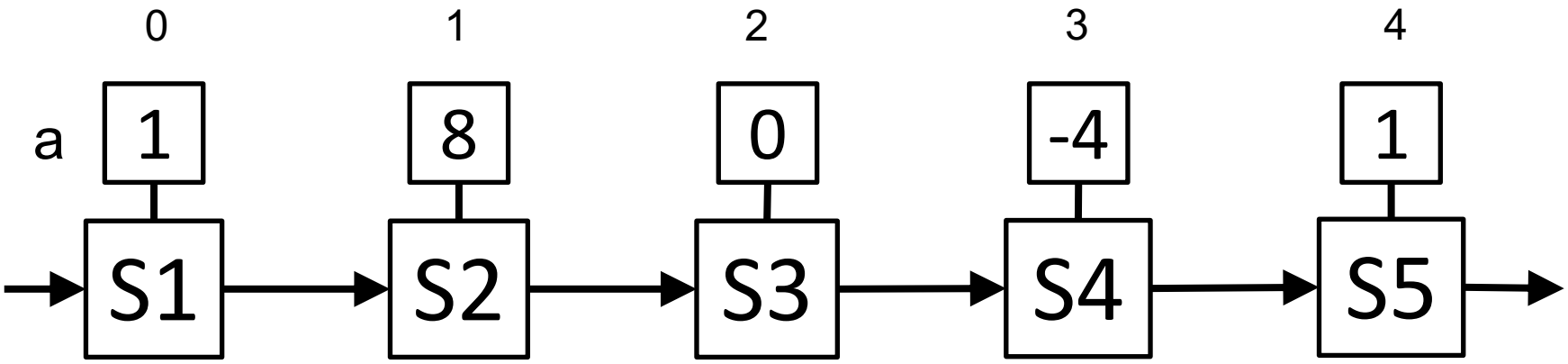
$1 + 8x_2 + (-4)x_2^3 + x_2^4$

$1 + 8x_3 + (-4)x_3^3 + x_3^4$

$1 + 8x_4 + (-4)x_4^3 + x_4^4$

$1 + 8x_5 + (-4)x_5^3 + x_5^4$

Polynomial calculation + Pipeline



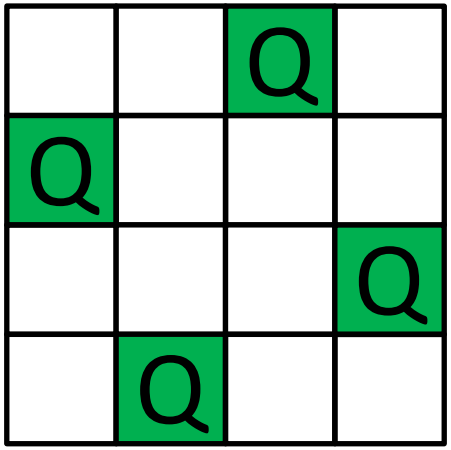
$$1 + 8x_3 + (-4)x_3^3 + x_3^4$$

$$1 + 8x_1 + (-4)x_1^3 + x_1^4 \qquad 1 + 8x_4 + (-4)x_4^3 + x_4^4$$

$$1 + 8x_2 + (-4)x_2^3 + x_2^4 \qquad 1 + 8x_5 + (-4)x_5^3 + x_5^4$$

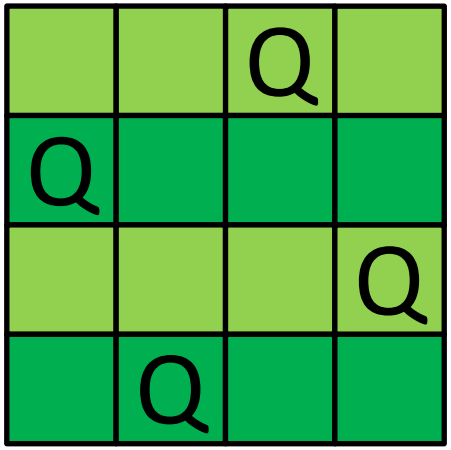
Aplicație: problema Damelor

N Queens Problem



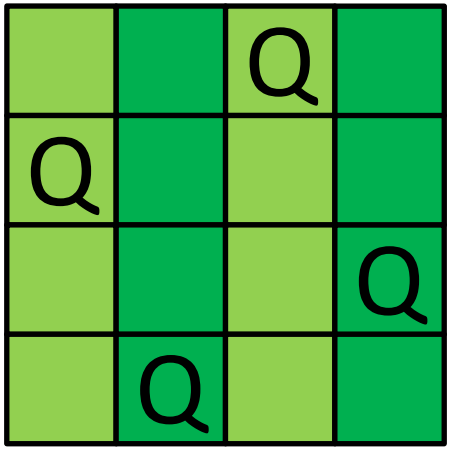
N Queens Problem

No more then one queen per line



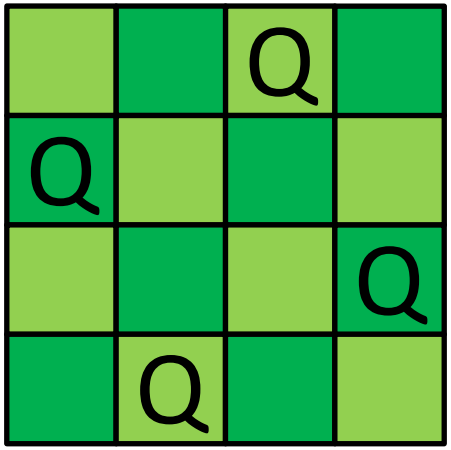
N Queens Problem

No more then one queen per column

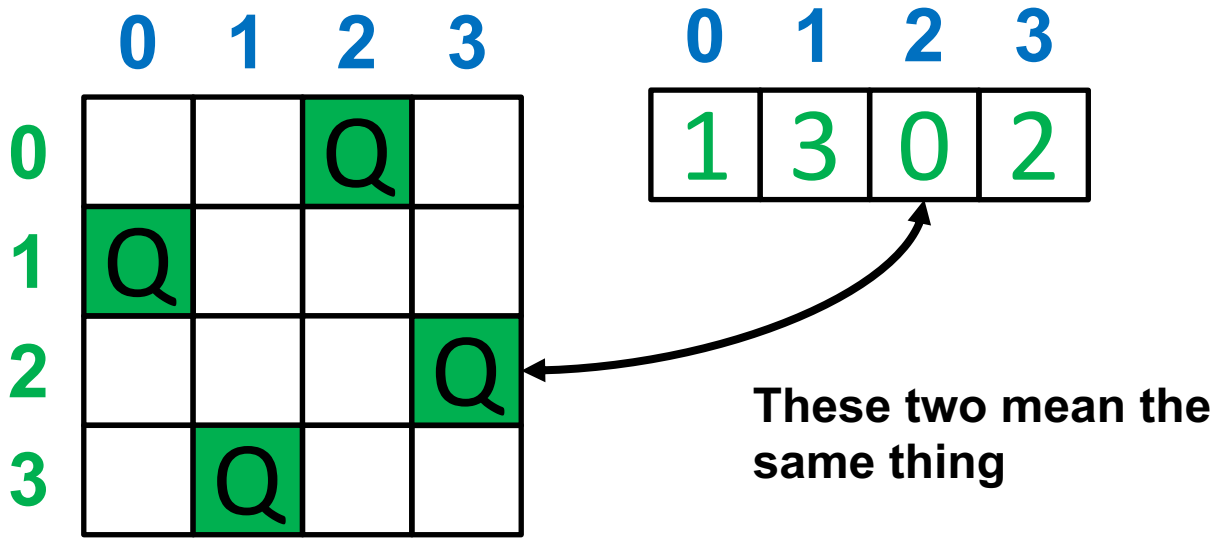


N Queens Problem

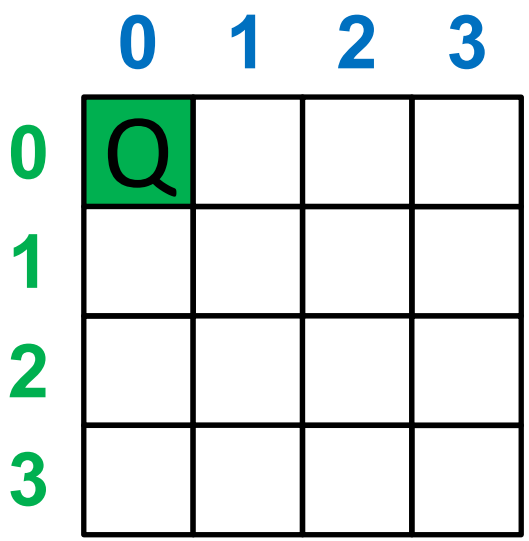
No more then one queen per diagonals

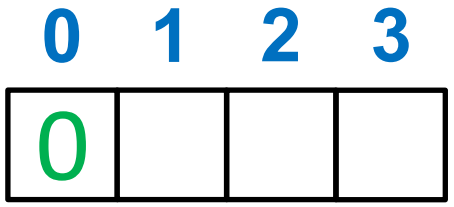


N Queens Problem

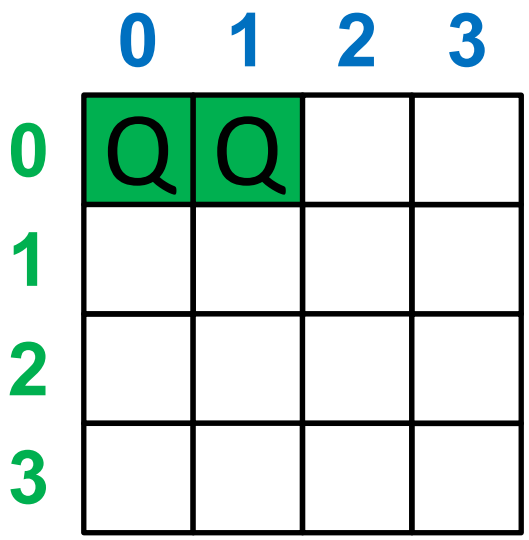


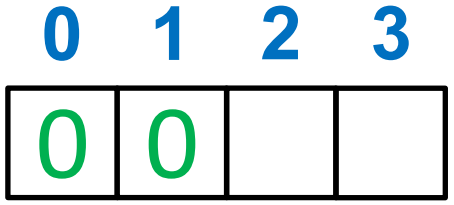
N Queens Problem - Solution





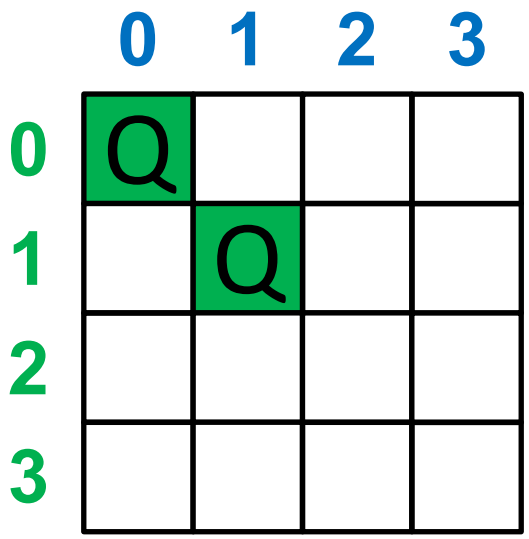
N Queens Problem - Solution

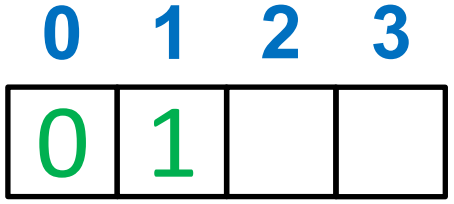




Line conflict

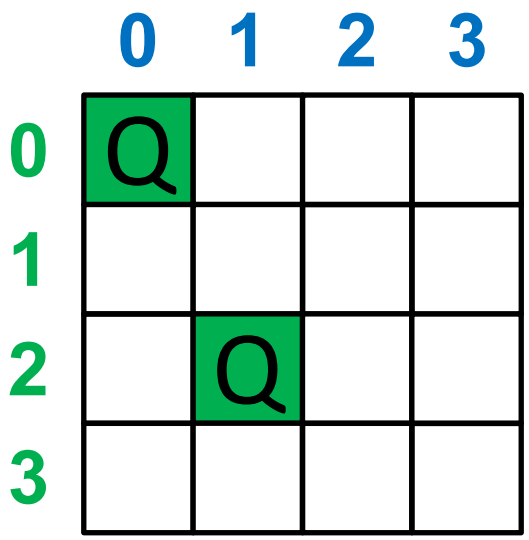
N Queens Problem - Solution





Diagonal conflict

N Queens Problem - Solution

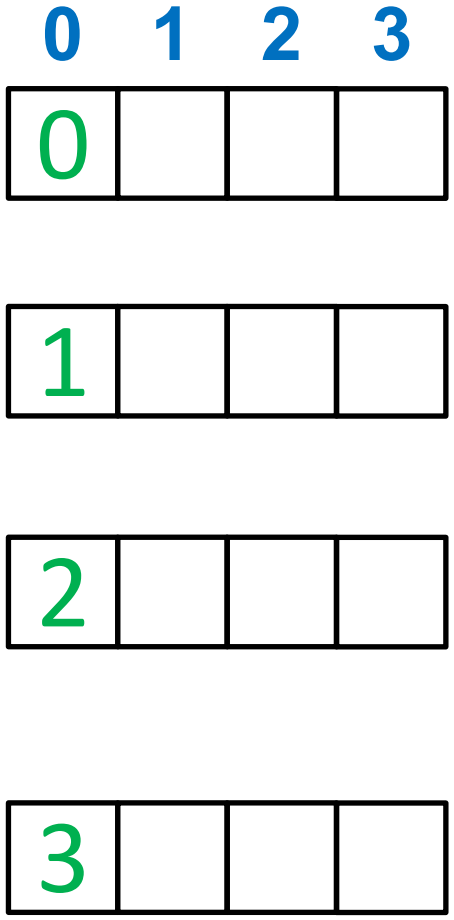


0	1	2	3
0	2		

OK.

And so on…

N Queens Problem – Parallel Solution



N Queens Problem – Parallel Solution

0	1	2	3	
0	0			x
0	1			x
0	2			
0	3			

0	1	2	3	
1	0			x
1	1			x
1	2			x
1	3			

0	1	2	3	
2	0			
2	1			x
2	2			x
2	3			x

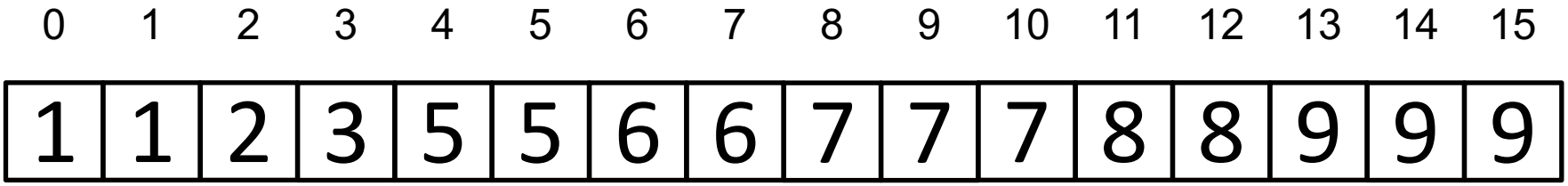
And so on…

0	1	2	3	
3	0			
3	1			
3	2			x
3	3			x

Cautare binara

Binary Search

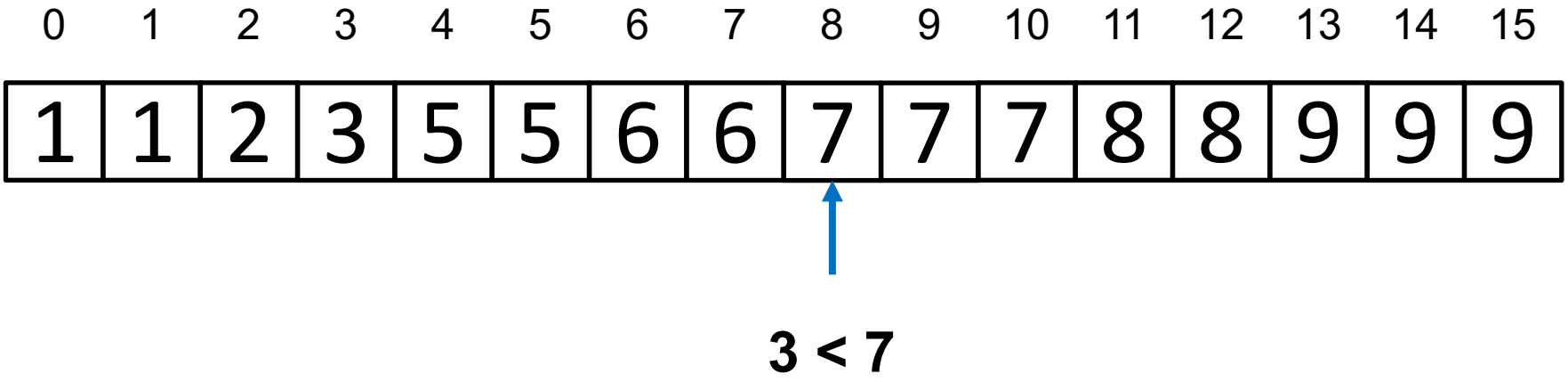
Searching for 3



Binary Search

Searching for | 3 |

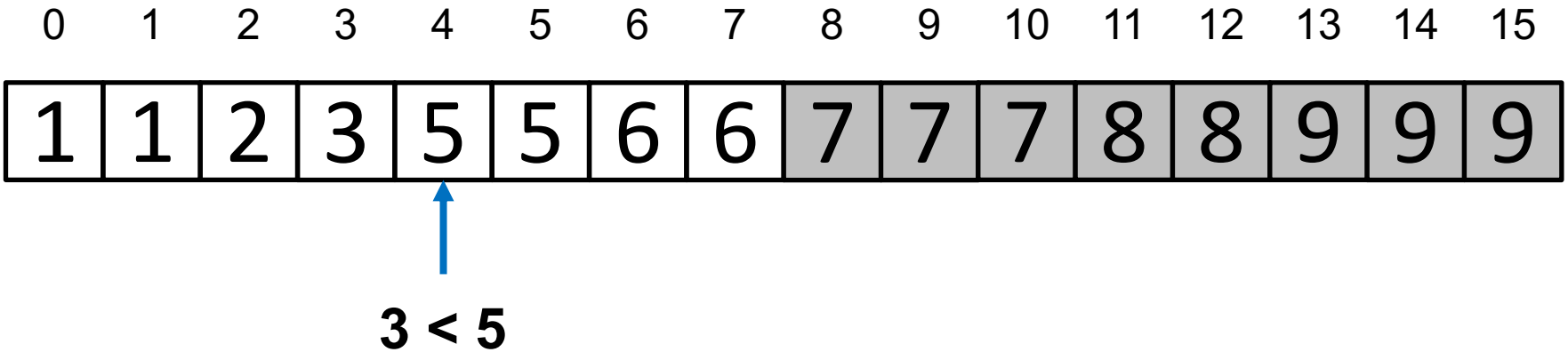
Interest area | 0 | 15 |



Binary Search

Searching for | 3 |

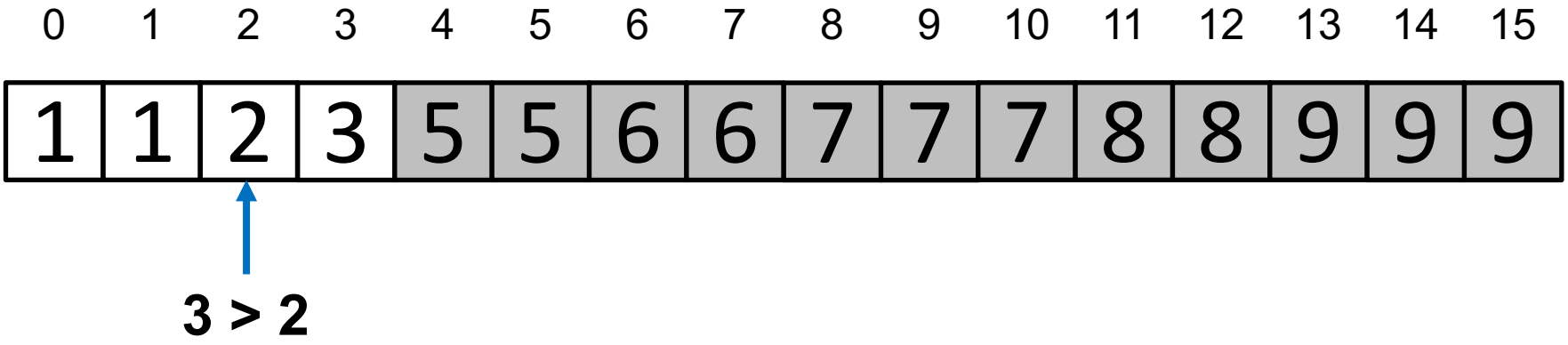
Interest area | 0 | 7 |



Binary Search

Searching for | 3 |

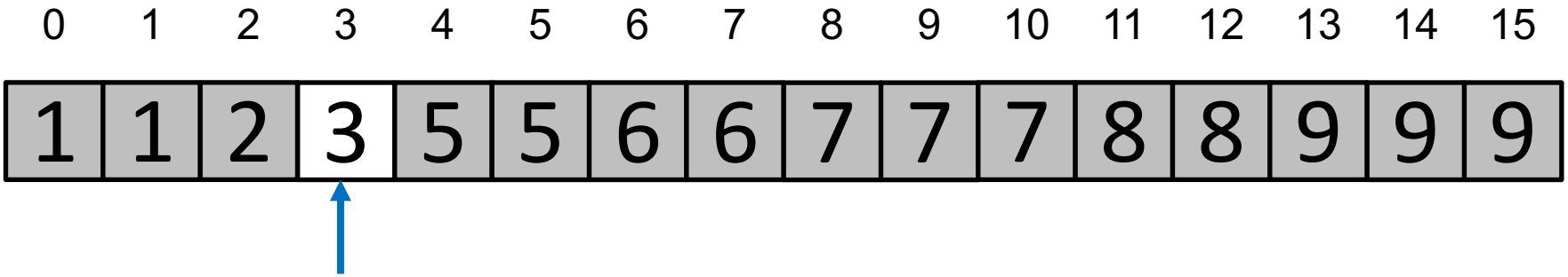
Interest area | 0 | 3 |



Binary Search

Searching for | 3 |

Interest area | 3 | 3 |



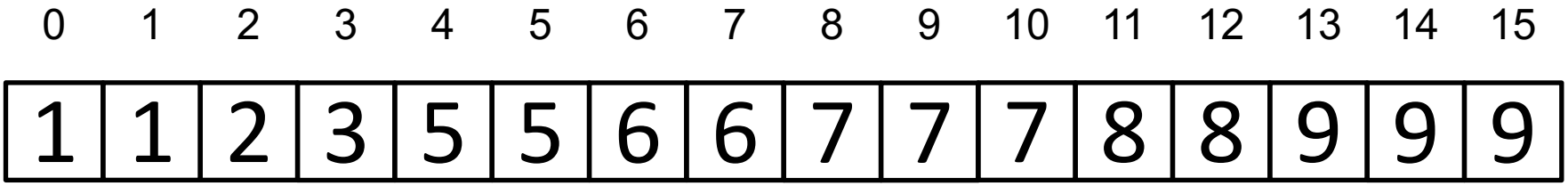
3 = 3

end

O($log_2(n)$) time

Parallel Search

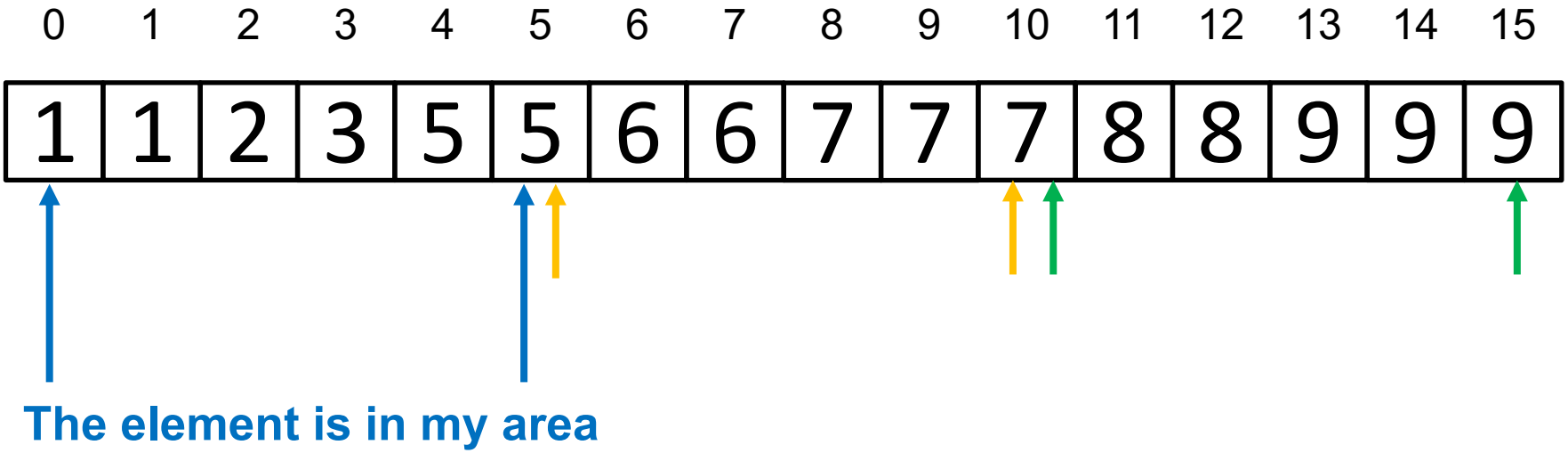
Searching for 3



Parallel Search

Searching for 3

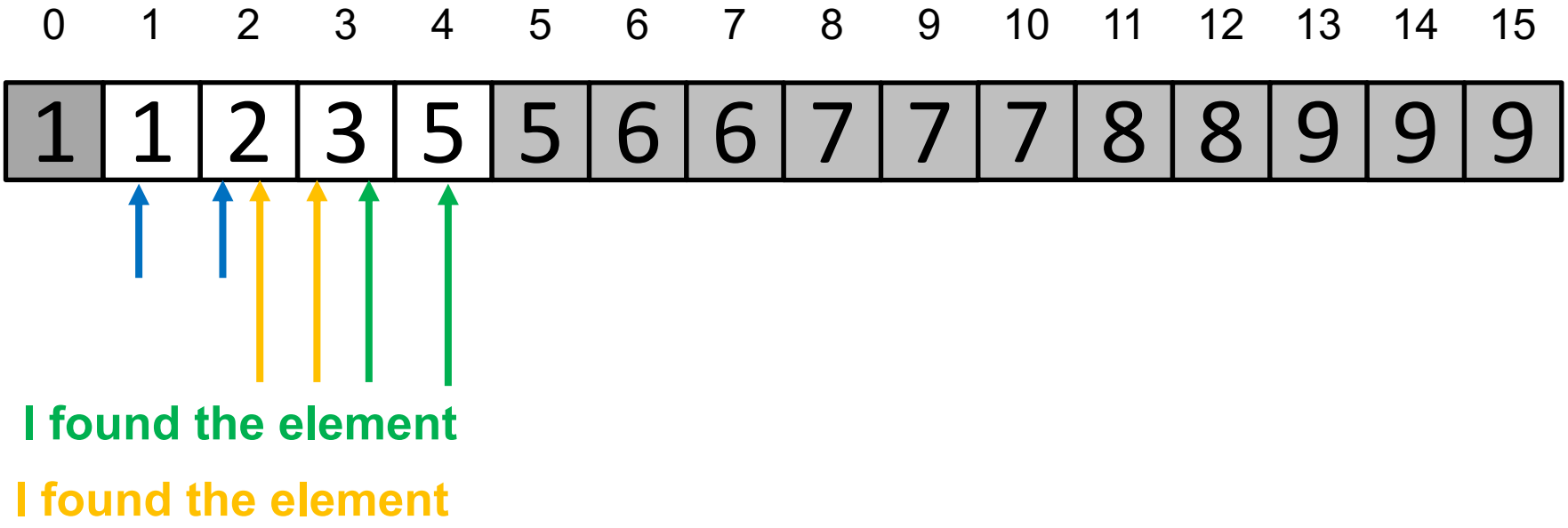
Interest area 0 15



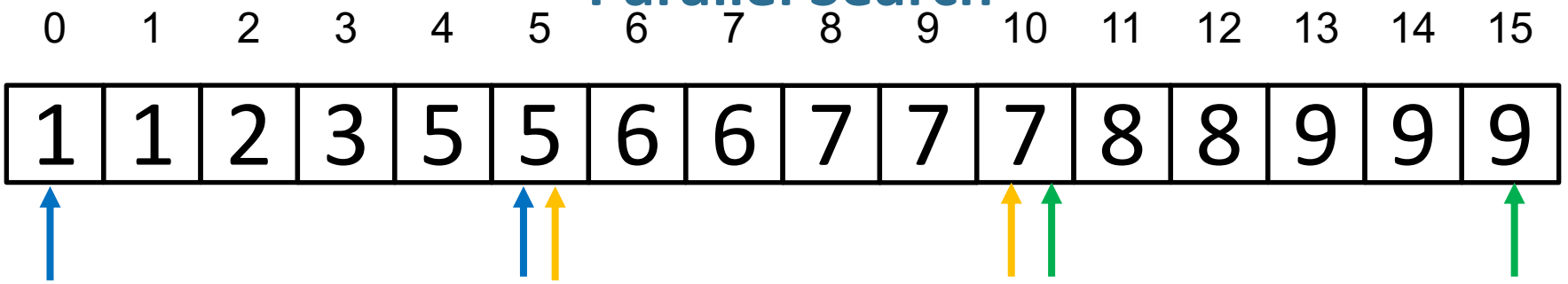
Parallel Search

Searching for 3

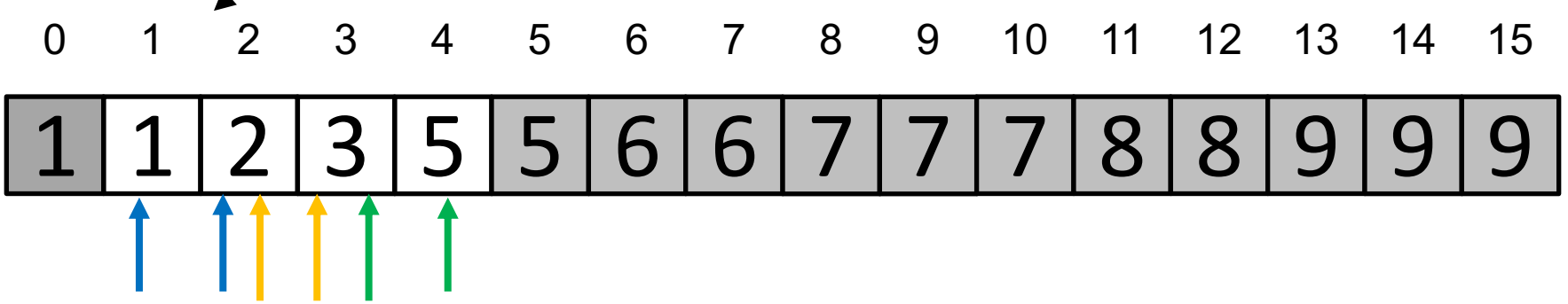
Interest area 1 4



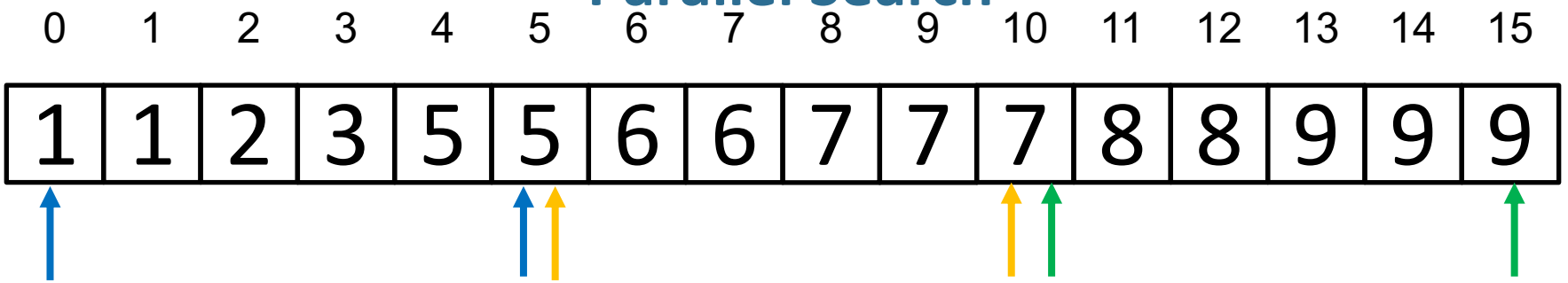
Parallel Search

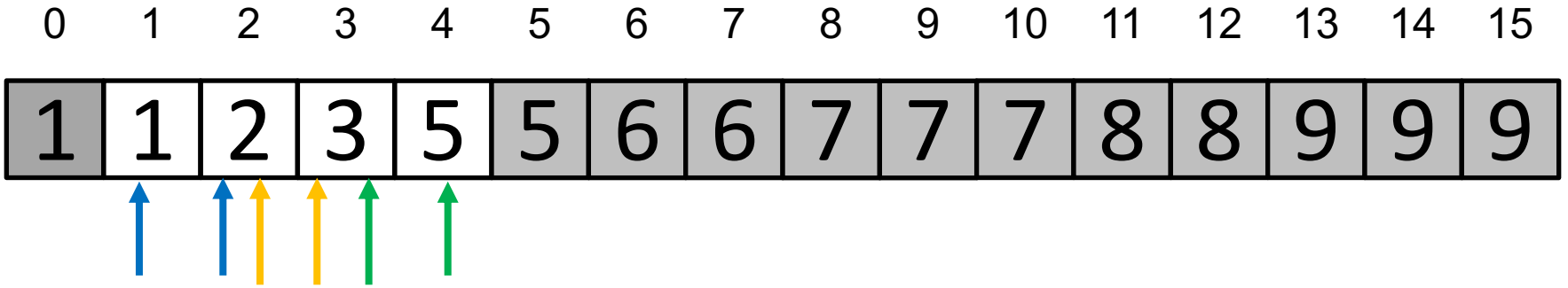


Operations can**NOT** be executed in parallel



Parallel Search



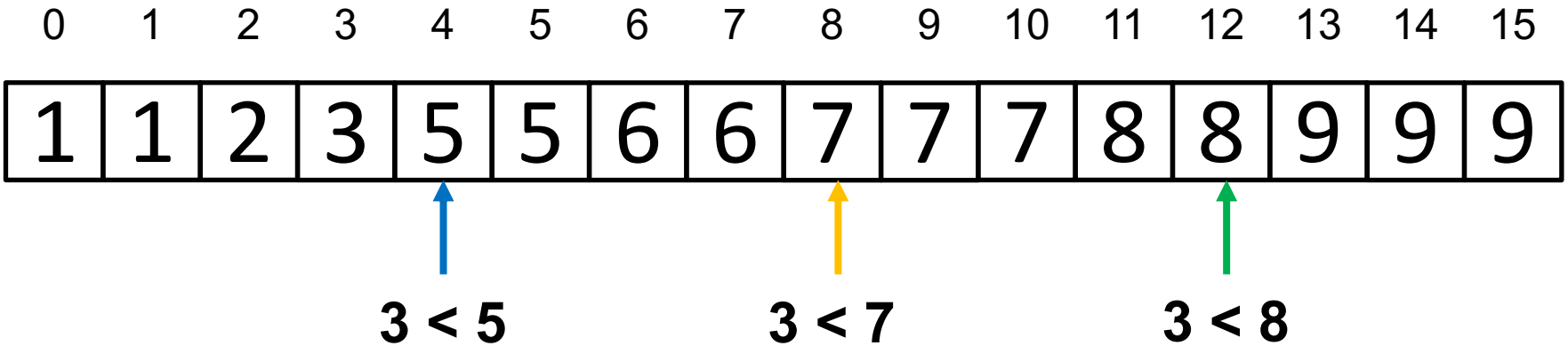


Parallel Search – solutia 2 (cu mai putine threaduri)

Parallel Search – solution 2 (fewer threads)

Searching for | 3 |

Interest area | 0 | 15 |

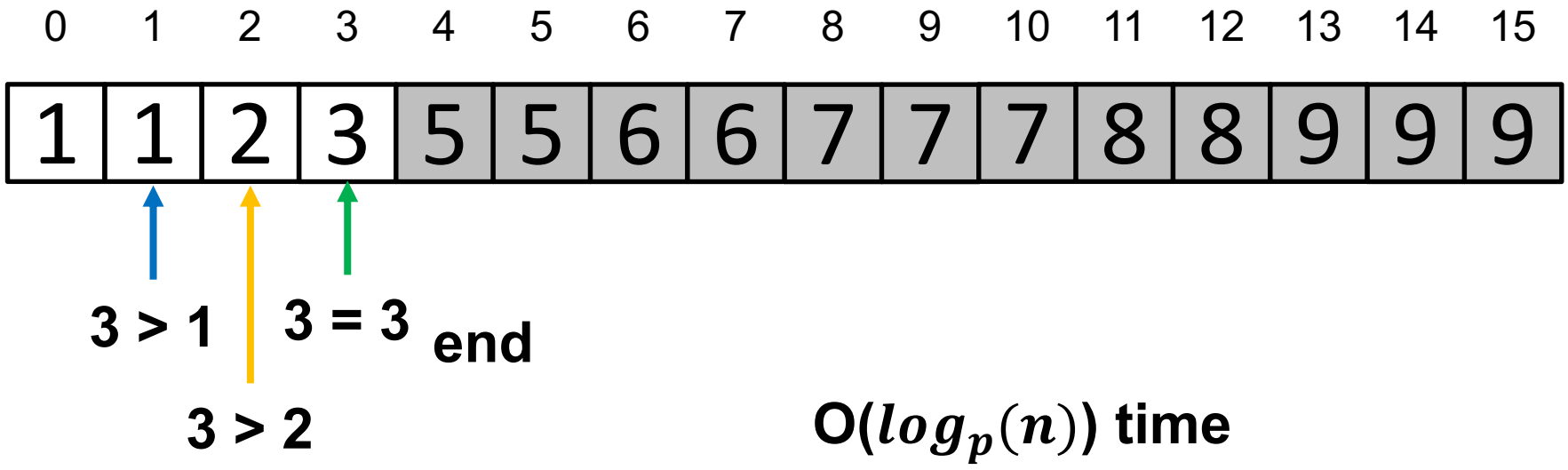


Operations can be executed in parallel

Parallel Search

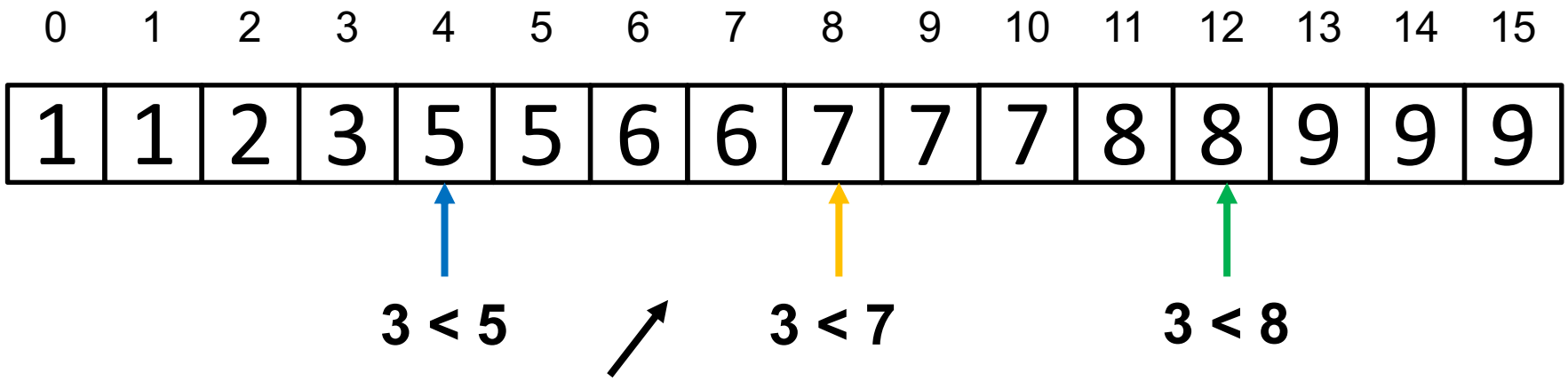
Searching for | 3 |

Interest area | 0 | 3 |

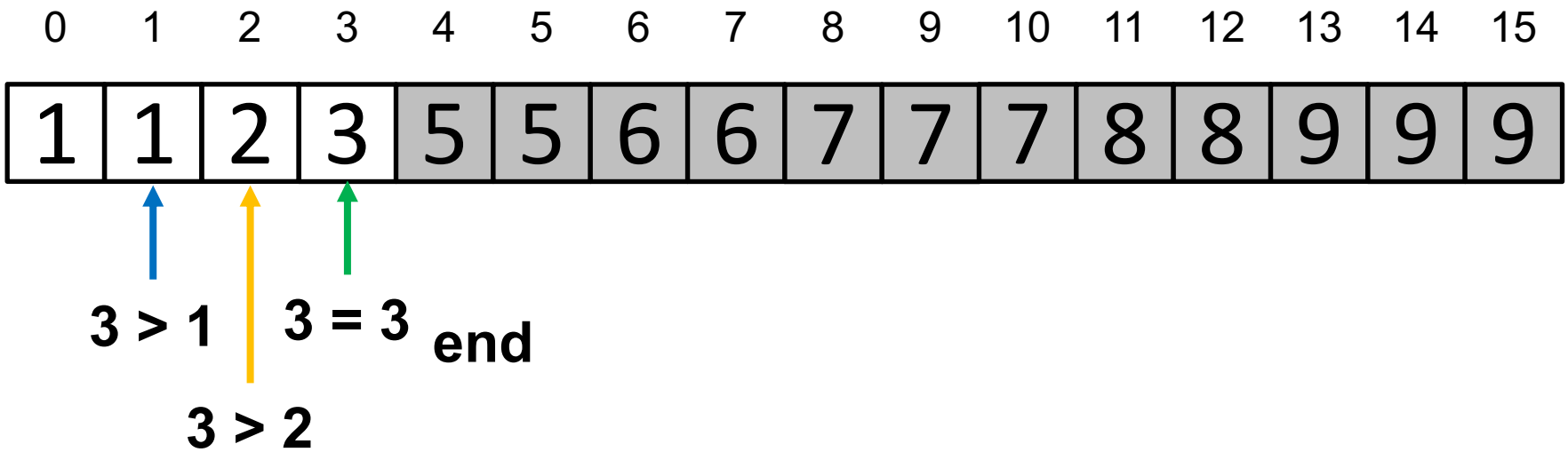


O($log_p(n)$) time

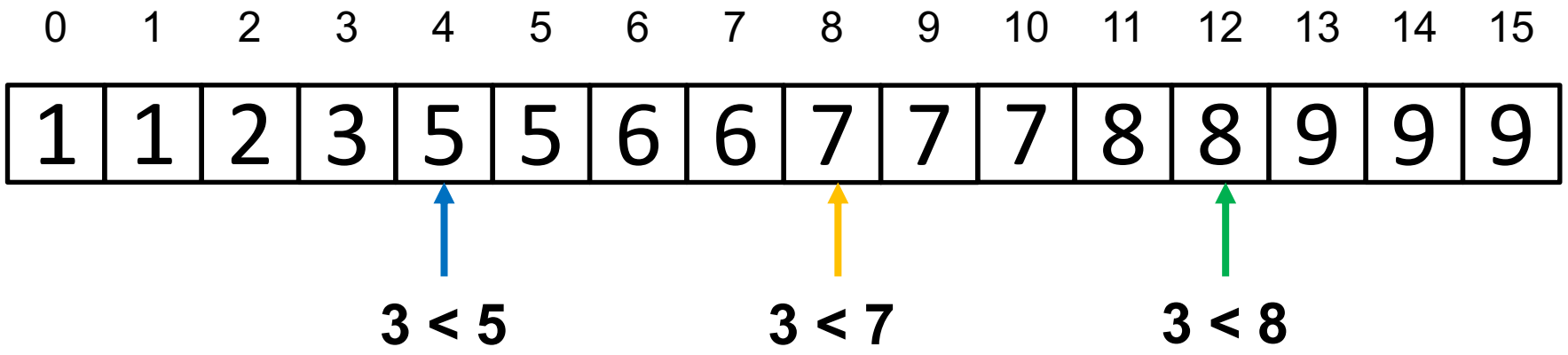
Parallel Search

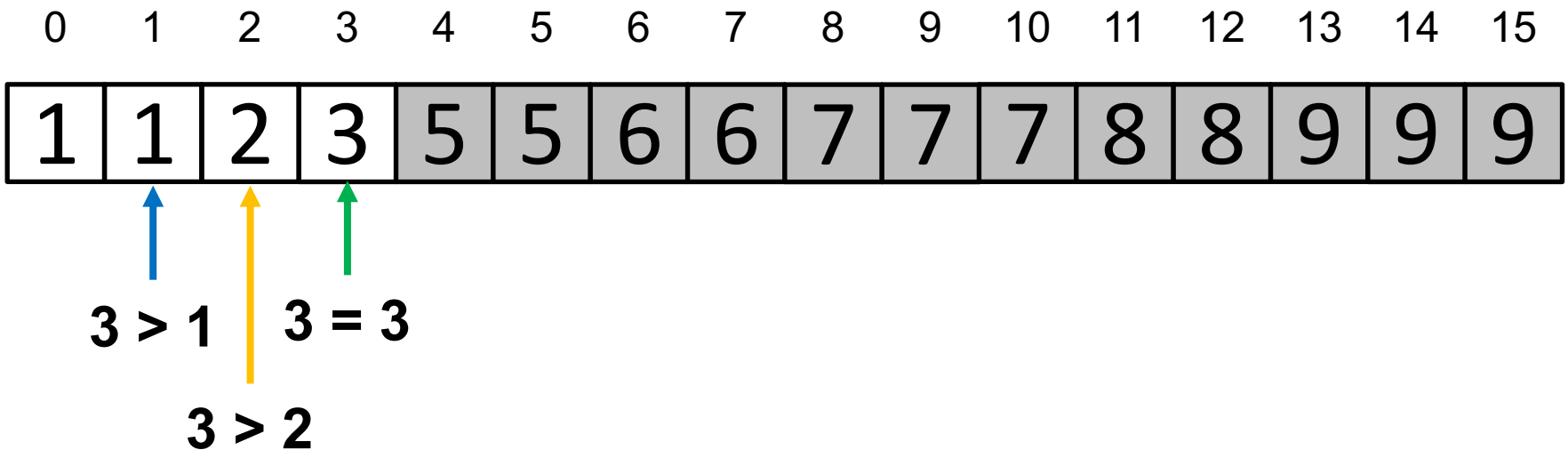


Operations can**NOT** be executed in parallel



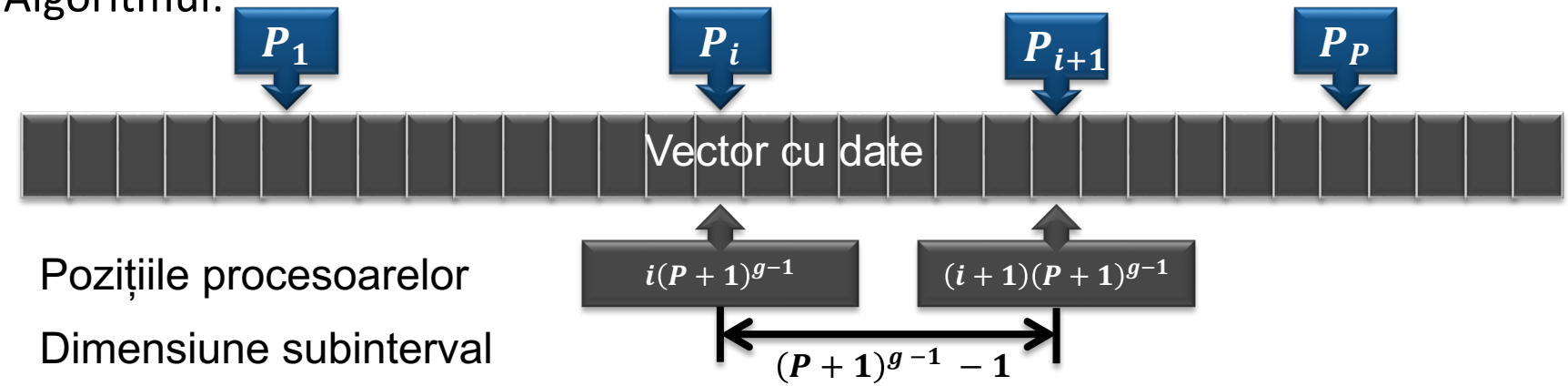
Parallel Search





Performanta?

- Algoritmul:

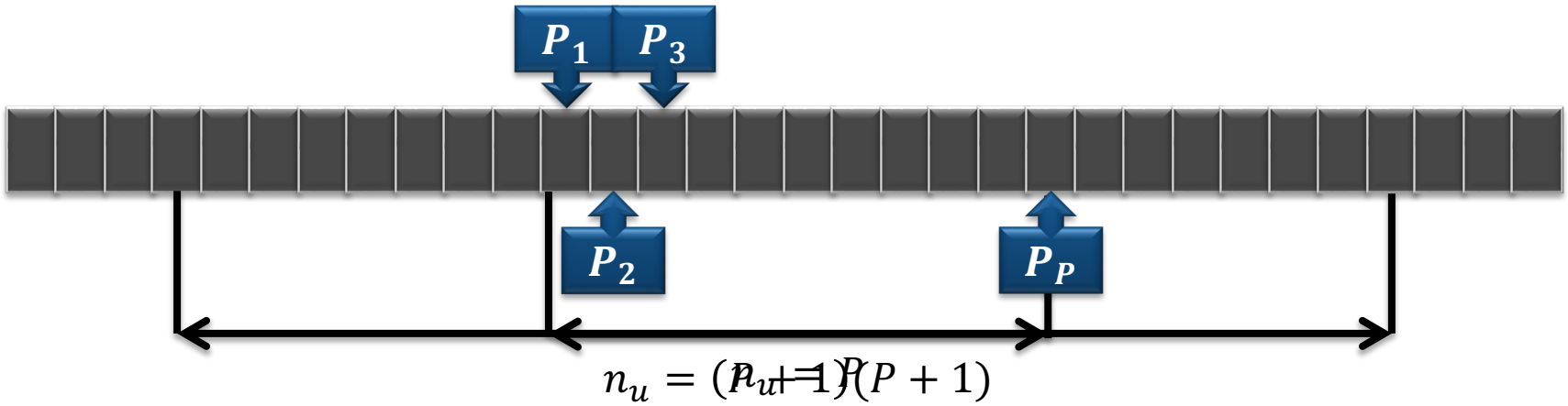


  - **g** : numărul de paşi
  - **P** : numărul de procesoare
  - **n** : numărul de elemente ale vectorului
  - P_i : procesorul cu indicele i

- Deoarece fiecare etapă se aplică unei secvenţe de dimensiune $\frac{1}{P+1}$ din secvenţa precedentă, sunt necesare:

$O(\log_{P+1}(n+1))$ etape.

Justificare



în ultimul pas P procesoare acoperă P elemente

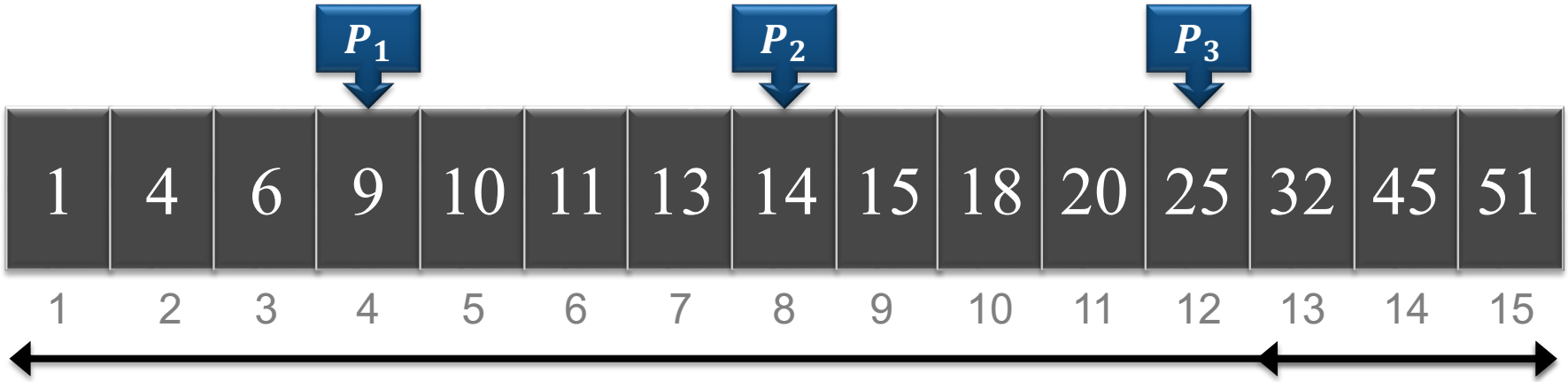
în pasul anterior $(P+1)^2 - 1$

în g paşi $(P+1)^g - 1$

din $(P+1)^g - 1 \geq n$ *(n = nr elemente şir)*

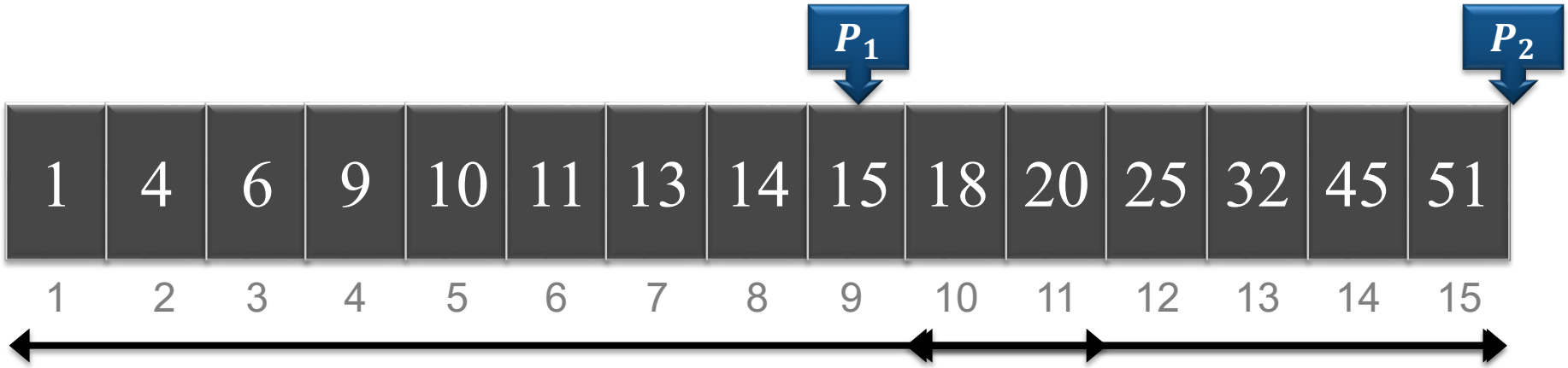
rezultă $\boldsymbol{g = sup\left(\frac{\log(n+1)}{\log(P+1)}\right)}$

Exemplu 1



- g = 2 (numărul căutat)
- P = 3 (numărul de procesoare)
- n = 15 (dimensiunea vectorului)
- Din $i(P+1)^{g-1} \Rightarrow P_1$ va ocupa poziția 4 din vector
  - $i = 2$
- Din $(P+1)^{g-1} \Rightarrow P_2$ va ocupa poziția 8 din vector
- Lungimea unei subsecvențe în primul pas este $(P+1)^{g-1} - 1 = \mathbf{3}$.
  - Din $i(P+1)^{g-1} \Rightarrow P_3$ va ocupa poziția 12 din vector

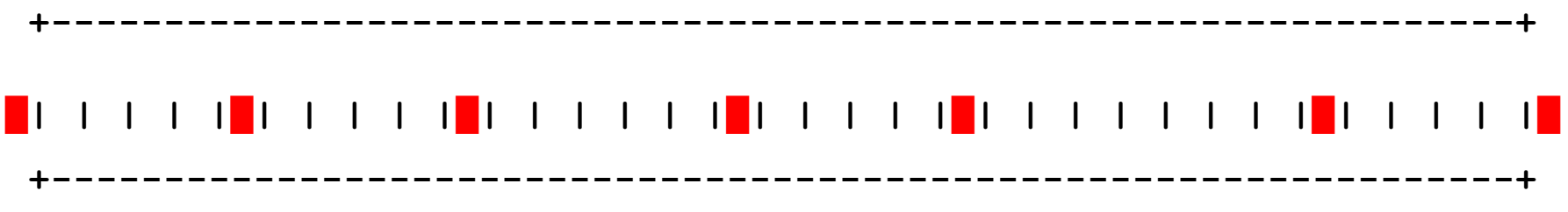
Exemplu 2



- X = 21 *(numărul căutat)*
- P = 2 *(numărul de procesoare)*
- n = 15 *(dimensiunea vectorului)*
- Din $i(P+1)^{g-1} \Rightarrow P_1$ va ocupa poziția 9 din vector
  - $i = 2$
- Din $(P+1)^{g-1} \Rightarrow g = 2$ P_2 va ocupa poziția **18** din vector
- Lungimea unei subsecvenţe în primul pas este $(P+1)^{g-1}-1 = \mathbf{8}$.

Algoritm

```
/1/  /* Initializarea indicilor secventei de cautat */
   int q = 1;              /* limita stg interval crt */
   int r = n;              /* limita dr interval crt */
/2/  /* Initializarea rezultatului k si a numarului de pasi g */
   int x, k = 0;                        /* x = numar cautat, k = pozitia lui in sir */
   int g = sup(log(n+1)/log(P+1))
   enum sens {stg,dr};
   sens c[0:P+1];  /*c[i] – x este la stg / dr pozitiei procesului i*/
    c[0] = dr; c[P+1] = stg;
    int j[1:P];                    /* j[i] = pozitia din sir inspectata de proces i*/
```

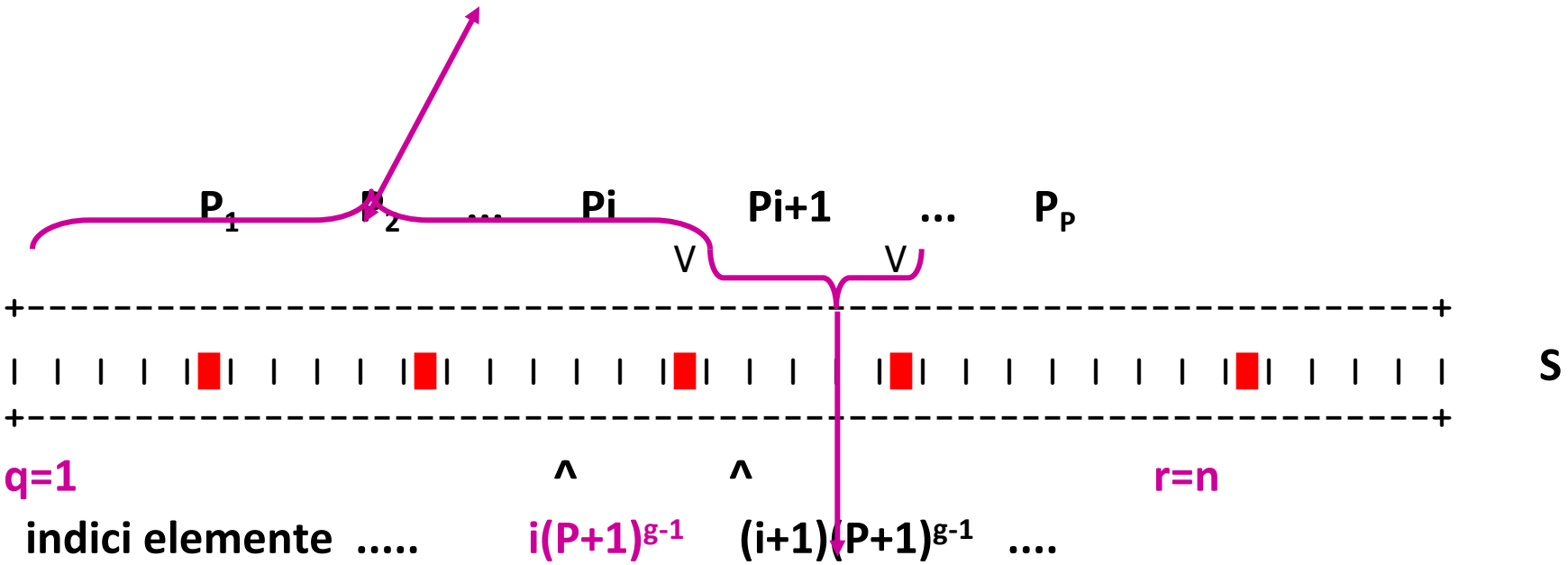


process P[i = 1 to P]{

/3/ gaseste pozitia k unde se afla valoarea x

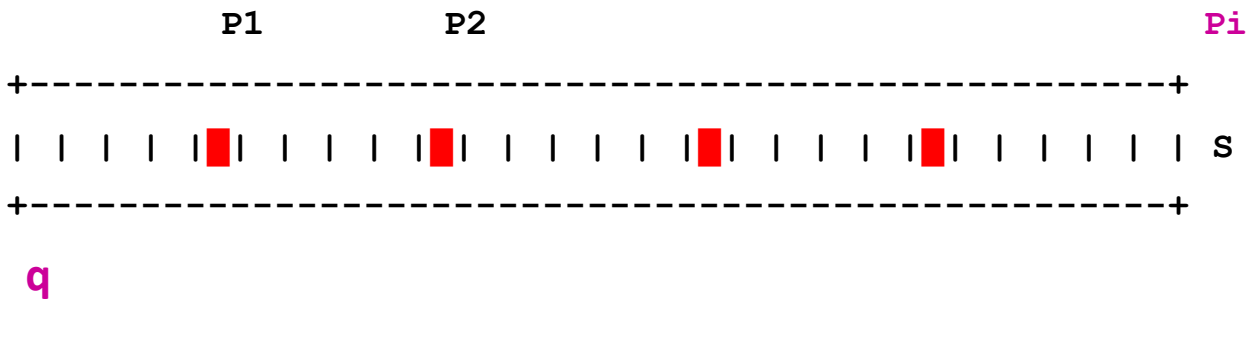
while (q<=r and k=0){ /* subsir exista si valoare negasita */

j[i] = (q-1) + $i*(P+1)^{g-1}$;



dimensiune subinterval $(P+1)^{g-1}-1$

```
while (q<=r and k=0){
    j[i] = (q-1) + i*(P+1)^(g-1)
/* Pi compara x cu s[j[i]] si determina partea de secv acceptata */
    if (j[i] <= r)                    /* pozitie in interior sir */
       if (s[j[i]] == x){ k = j[i];          /*pozitie gasita*/
       else if (s[j[i]] > x) c[i] = stg;
       else c[i] = dr;
    else {j[i] = r+1;                 /*j[i] indica o pozitie afara din sir*/
          c[i] = stg;}
```



```
/* calculeaza indicii subsecventei urmatoare */
/* doar un proces poate face modificarea intervalului de cautare */
        if (c[i] <> c[i-1]){
                r = j[i] – 1;
             if (i>1) q = j[i-1] + 1;
        }
/* doar procesul P executa operatia urmatoare*/
        if (i==P and c[i]<>c[i+1]) q = j[i] +1;
        g := g-1;
   }                    /*end while*/
}        /*end process*/
```

Observaţii

- Complexitatea

$$t(\mathrm{n}) = \mathrm{O}(\log_{P+1}(n+1)) \rightarrow$$ algoritmul este *timp* optimal

- Costul

$$O(P\ \log_{P+1}(n+1)) \rightarrow$$ algoritmul nu este *cost* optimal

- Notă
  - cerinţa este ca toate elementele secvenţei S să fie distincte

- În cazul în care șirul conține valori egale
  - Menţinerea eficienţei algoritmului → model CRCW.

Proprietati (dorite) ale algoritmilor paraleli

Număr de procesoare

- p(n) < n - uzual p(n) este funcţie sublineară de n
- p(n) adaptiv - algoritmul sa nu depindă de un număr fix de procesoare

Timp de execuţie

- t(n) mic - semnificativ mai mic decat al variantei secvenţiale
- t(n) adaptiv - invers proportional cu p(n)

Cost

- c(n) sa fie minim, deci algoritmul sa fie cost-optimal

Reminder

Costul unui algoritm se definește ca produsul dintre numărul de procesoare folosite și timpul de rulare.
Pe o mașină secvențială, P=1, deci costul algoritmul este timpul (asimptotic) de execuție.

Un algoritm paralel cost optimal este unul pentru care costul este θ (timpul de execuție secvențial).
Dacă un algoritm paralel nu este cost optimal, el poate fi făcut cost optimal fie reducând timpul asimptotic de execuție, fie reducând numărul de procesoare.

Fie timpul de execuție al algoritmului secvențial pentru efectuarea unor task-uri de dimensiune n, $\theta(n)$, și timpul de execuție a versiunii paralele a acelui algoritm $\theta(\lg n)$, acesta folosind n procesoare, fiecare păstrând un element de date. Costul este așadar:

$$cost = n\theta(\lg n) = \theta(n \lg n)$$

Costul acesta nu este optimal. Pentru a obține un algoritm de cost optimal, putem fie reduce timpul de execuție la $\theta(1)$, fie reduce numărul de procesoare (la $n/\lg n$, astfel încât produsul $n/\lg n$ cu $\theta(\lg n)$ să dea $\theta(n)$).

Să presupunem acum că fiecare procesor deține $\lg n$ date. Dacă procesul secvențial este liniar cu numărul de elemente, atunci fiecare dintre cele $n/\lg n$ procesoare va procesa în paralel cele $\lg n$ date locale într-un timp de $\theta(\lg n)$. Cele $n/\lg n$ procesoare vor efectua algoritmul (non-cost optimal) anterior asupra celor $n/\lg n$ rezultate din pasul secvențial într-un timp de $lg(n/\lg n) = \lg n - lg(\lg n) = \theta(\lg\ n)$. Costul fiecărui pas este $\theta(\lg n)$, dar costul final este $(n/\lg n)\theta(\lg n) = \theta(n)$.